BRAIN TUMOR DETECTION WITH ARTIFICIAL INTELLIGENCE

A Project Report
Submitted by

VEDIKA SONI
2303031241378

SAPPA KAVYA SRI
2303031241210

BERI PRANAV
2303031241579

TATIPARTHI SAI PRANEETH REDDY
2303031241328

In Partial Fulfilment for the
Award of the Degree of
BACHELOR OF TECHNOLOGY
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Under the Guidance of
Deepika Pandey
Professor



PARUL UNIVERSITY, VADODARA
October - 2026

PARUL UNIVERSITY

Certificate

This is to certify that the project entitled “Brain Tumor Detection with Artificial Intelligence” of Group No. [ASMP-28] has been completed by

- VEDIKA SONI – 2303031241378
- SAPPA KAVYA SRI – 2303031241210
- BERI PRANAV – 2303031241579
- TATIPARTHI SAI PRANEETH REDDY – 2303031241328

under my guidance, in partial fulfilment of the Bachelor of Technology (B.Tech.) in Artificial Intelligence and Data Science at Parul University in the Academic Year 2026 - 2027.

Date of Submission:

Project Guide
Deepika Pandey
Professor

Head of Department
Dr. Sanjay Agal

Project Coordinator
Dr. Mohammad Arif

External Examiner

Department of Artificial Intelligence and Data Science
Parul Institute of Engineering and Technology
Parul University, Vadodara

Acknowledgements

We would like to express our sincere gratitude to everyone who supported us throughout the development of this project.

We are thankful to our project guide, Deepika Pandey, for the continuous guidance, valuable suggestions, and encouragement provided throughout the course of this project. We are also grateful to the Head of the Department of Artificial Intelligence and Data Science, Parul Institute of Engineering and Technology, for the support extended to us.

We would also like to thank our families and friends for their constant encouragement and support during the course of this project.

Finally, we thank all the individuals who, directly or indirectly, contributed towards the successful completion of this project.

VEDIKA SONI
SAPPA KAVYA SRI
BERI PRANAV
TATIPARTHI SAI PRANEETH REDDY
B.Tech. – Artificial Intelligence and Data Science
Parul University, Vadodara

Abstract

Brain tumours are among the most serious neurological conditions, and early, accurate identification of the tumour type is central to treatment planning. Magnetic Resonance Imaging (MRI) is the preferred modality, but reading scans is time-consuming and depends on specialist experience. This project presents Brain Tumor Detection with Artificial Intelligence — a full-stack web application that classifies a brain MRI slice into one of four classes (glioma, meningioma, pituitary tumour or no tumour), reports the model's confidence for every class, and overlays a Grad-CAM heatmap that shows which region of the scan drove the decision.

The system combines a React 19 frontend with a Flask REST API and a PyTorch inference layer. Two interchangeable model back-ends sit behind the same API: a two-stage VGG16 transfer-learning baseline and an EfficientNet-B0 network that is fine-tuned end to end. Every scan is passed through a brain-region crop, resized to 224×224 pixels and normalised with ImageNet statistics before classification. The EfficientNet-B0 model was obtained by fine-tuning an existing checkpoint with weighted cross-entropy, label smoothing, AdamW with a one-cycle learning-rate schedule and strong data augmentation, using a validation split carved out of the training folder so that the test set stayed untouched.

On the 1,311-image test set of the public Kaggle Brain Tumor MRI dataset the fine-tuned model reaches 99.85% accuracy (two errors), compared with 97.10% for the VGG16 baseline and 80.8% for the starting checkpoint, whose glioma recall of 18% rose to 99.7%. Passing the whole test set through the live API reproduced the offline result exactly. Eighteen automated backend tests pass, and a latency benchmark of 100 requests measured a 95th-percentile response time of 197 ms for one user and 409 ms with four concurrent users on a CPU-only laptop. The work also uncovered and fixed a thread-safety defect in the inference path. The application is an academic prototype and is not a medical device.

Table of Contents

Acknowledgements ..... **iii**

Abstract ..... **iv**

Table of Contents ..... **v**

List of Tables ..... **viii**

List of Figures ..... **ix**

Chapter 1 Introduction **1**

1.1 Background of the Study 1
1.2 Brain Tumours and Their Types 1
1.3 Role of MRI in Brain Tumour Detection 2
1.4 Limitations of Manual Diagnosis 3
1.5 Role of Artificial Intelligence in Medical Imaging 3
1.6 Convolutional Neural Networks and Transfer Learning 4
1.7 Motivation of the Project 4
1.8 Problem Statement 5
1.9 Objectives of the Project 5
1.10 Scope of the Project 6
Included in Scope 6
Excluded from Scope 6
1.11 Key Features of the Proposed System 7
1.12 Expected Outcomes 7
1.13 Organisation of the Report 8

Chapter 2 Literature Survey **9**

2.1 Introduction to Existing Systems 9
2.2 Study of Deep-Learning Approaches for Brain Tumour Detection 9
2.2.1 Segmentation and Edge-Based Methods 9
2.2.2 Autoencoder-Based Approaches 9
2.2.3 CNN Classification Networks 10
2.2.4 Detection-Based Approaches 10
2.3 Overview of Transfer-Learning Architectures 11
2.3.1 VGG16 11
2.3.2 ResNet and MobileNet 11
2.3.3 EfficientNet 11
2.4 Explainable AI and Grad-CAM 11
2.5 Public Datasets for Brain Tumour Research 12
2.6 Comparative Analysis of Existing Approaches 12
2.7 Research Gaps Identified 13
2.8 Summary of Literature Survey 13

Chapter 3 Analysis / Software Requirements Specification (SRS) **15**

3.1 Introduction 15
3.2 Purpose of the System 15
3.3 Intended Users 15
3.4 Product Scope 15
3.5 MRI Image Input Processing 16
3.6 Tumour Classification using Deep Learning 16

3.7 Explainability Mechanism (Grad-CAM)....16
3.8 Result Display System....16
Functional Requirements Summary....17
3.9 Security and Privacy Requirements....17
3.10 Performance Requirements....18
3.11 Reliability and Availability....18
3.12 Scalability....18
Non-Functional Requirements Summary....18
3.13 Hardware Requirements....19
3.14 Software Requirements....19
3.15 User Interface Requirements....19
3.16 Constraints and Assumptions....20
3.17 References Used in Analysis....20
Chapter 4 System Design....21
4.1 Introduction....21
4.2 Overall System Architecture....21
4.3 Use Case Diagram....23
4.4 Class Diagram....23
4.5 Sequence Diagram....25
4.6 Data Flow of the System....26
4.7 Frontend Design (React UI Structure)....26
4.8 Backend Design (Flask Architecture)....27
4.9 Model Design....29
4.10 Grad-CAM Design....29
4.11 Image Pre-processing Design....30
4.12 Security Considerations....30
4.13 Performance Optimisation....30
4.14 Integration Strategy....31
Chapter 5 Methodology....32
5.1 Development Methodology (Iterative / Agile Approach)....32
5.2 Project Planning Strategy....33
5.3 Requirement Gathering Process....33
5.4 Dataset Description and Preparation....33
5.5 Data Pre-processing and Augmentation....35
5.6 Model Development Strategy....36
5.6.1 Stage 1: VGG16 Baseline....36
5.6.2 Stage 2: EfficientNet-B0 First Run....36
5.6.3 Stage 3: Diagnosis and Fine-tuning of the Checkpoint....36
5.7 Frontend Development Strategy....37
5.8 Backend Development Strategy....37
5.9 Tools Used....38
5.10 Testing Strategy....38
5.11 Risk Management....39
5.12 Gantt Chart....39
Chapter 6 Implementation....41
6.1 Introduction....41
6.2 System Setup and Environment....41
6.3 Dataset Loading and Pre-processing Implementation....42
6.4 Model Implementation....42

6.5 Fine-tuning Implementation....43
6.6 Grad-CAM Implementation....45
6.7 API Development....46
6.8 Thread-Safety of Inference....47
6.9 UI Development using React....48
6.10 Flowcharts for System Logic....48
6.11 Screenshots of Application....49
6.12 Key Code Snippets....54
Chapter 7 Testing....56
7.1 Introduction....56
7.2 Testing Strategy....56
7.3 Unit Testing....56
7.4 Integration Testing....57
7.5 System Testing....57
7.6 Functional Test Cases....58
7.7 Invalid Input and Robustness Testing....59
7.8 Concurrency Testing and the Thread-Safety Defect....60
7.9 Performance Testing....60
7.10 Overfitting and Generalisation Analysis....61
7.11 Results and Analysis....62
7.12 Screenshots of Testing Outcomes....64
Chapter 8 Conclusion and Future Work....67
8.1 Summary of the Project....67
8.2 Achievements....67
8.3 Comparison of Objectives vs Outcomes....68
8.4 Current Limitations....68
8.5 Future Work — External Validation and Data Diversity....69
8.6 Future Work — Multi-sequence, 3-D Analysis and Segmentation....69
8.7 Future Work — Reliability, Calibration and Out-of-distribution Detection....69
8.8 Future Work — Clinical Integration and Deployment....70

List of Tables

Table 2.1: Comparison of Existing Approaches....12
Table 3.1: Functional Requirements of the System....17
Table 3.2: Non-Functional Requirements of the System....18
Table 3.3: Hardware and Software Requirements....19
Table 5.1: Project Milestones and Timeline....33
Table 5.2: Composition of the Dataset....34
Table 5.3: Fine-tuning Configuration of the Final Model....37
Table 5.4: Technology Stack Overview....38
Table 5.5: Risk Register and Mitigation Strategies....39
Table 6.1: Module-wise Description of the System....41
Table 7.1: Automated Test Suite (pytest)....56
Table 7.2: Per-class Results of the Final Model on the Test Set....57
Table 7.3: Functional Test Cases....58
Table 7.4: Invalid-Input and Robustness Test Results....59
Table 7.5: Performance Testing Results (/api/predict, 100 requests per scenario)....60
Table 7.6: Fine-tuning Progress (selected epochs)....61
Table 7.7: Comparison of Expected vs. Actual Results across Models....62
Table 8.1: Comparison of Objectives vs Outcomes....68

List of Figures

Fig 4.1: System Architecture — Brain Tumor Detection with AI.....22
Fig 4.2: Use Case Diagram.....23
Fig 4.3: Class Diagram.....24
Fig 4.4: Sequence Diagram — Upload a Scan and Receive a Diagnosis.....25
Fig 4.5: Data Flow Diagram — Level 1.....26
Fig 4.6: Backend Processing Flow — Flask + EfficientNet-B0.....28
Fig 4.7: Model Design — Inference Pipeline with Grad-CAM Branch.....29
Fig 5.1: Project Development Lifecycle — Iterative / Agile Model.....32
Fig 5.2: Dataset Class Distribution.....34
Fig 5.3: Sample MRI Slices from the Dataset (one row per class).....35
Fig 5.4: Technology Stack Overview.....38
Fig 5.5: Project Gantt Chart — 15-Week Development Timeline.....40
Fig 6.1: Execution Flow of the Fine-Tuning Script.....44
Fig 6.2: Scan Processing Pipeline — Prediction Decision Flow.....49
Fig 6.3: Home Page — Hero Section with Animated 3D Brain and Test-set Accuracy.....50
Fig 6.4: Upload Panel — Scan Selected and Previewed Before Analysis.....50
Fig 6.5: Glioma Result with Grad-CAM Heatmap and Class Probabilities.....51
Fig 6.6: Same Result in the Original Image View.....51
Fig 6.7: Meningioma Result (confidence 94.4%).....52
Fig 6.8: Pituitary Tumour Result (confidence 96.7%).....52
Fig 6.9: No-Tumour Result (confidence 97.0%).....53
Fig 6.10: Session History after Analysing Five Scans.....54
Fig 6.11: Error Message Shown When the Server Cannot Read the Uploaded File.....54
Fig 7.1: Confusion Matrix of the Fine-tuned EfficientNet-B0 on the Test Set.....58
Fig 7.2: Precision, Recall and F1-score per Class (Test Set).....58
Fig 7.3: /api/predict Latency — One User vs. Four Concurrent Users.....61
Fig 7.4: Training Curves of the Fine-tuned Model (accuracy and worst-class recall).....62
Fig 7.5: Per-Class Recall and Overall Accuracy — Model Comparison.....63
Fig 7.6: Grad-CAM Examples: Original MRI (top) and Model Attention Heatmap (bottom)63
Fig 7.7: Automated Backend Test Run — 18 Tests Passing.....64
Fig 7.8: Full Test Set through the Live API — 99.85% Accuracy.....64
Fig 7.9: Backend Log while Serving Requests.....65
Fig 7.10: Health Endpoint (/api/health) in the Browser.....65
Fig 7.11: Model-information Endpoint (/api/model-info) in the Browser.....66

Chapter 1
Introduction

1.1 Background of the Study

Medical imaging has become the backbone of modern diagnosis. Over the last five decades, techniques such as X-ray radiography, computed tomography (CT), ultrasound and magnetic resonance imaging (MRI) have moved from specialised research tools to routine instruments in almost every hospital. Each year these systems generate an enormous volume of images, and every image must be examined by a trained specialist before a clinical decision is made. The number of scans has grown faster than the number of radiologists, so the interpretation of images is increasingly a bottleneck in the path from symptom to treatment.

Neuro-imaging is a particularly demanding part of this workload. The brain is the control centre of the body, its anatomy is intricate, and abnormal tissue can be small, faint or located close to critical structures. A brain tumour that is detected early and correctly classified can often be treated with a less invasive procedure and a better outcome than one that is found late. The choice of treatment — surgery, radiotherapy, chemotherapy or careful observation — depends strongly on the type of tumour, which makes accurate classification as important as detection itself.

Over the same period, artificial intelligence has changed how computers deal with images. Deep convolutional neural networks (CNNs), trained on large collections of labelled pictures, now match or exceed human performance on several visual recognition benchmarks. When these networks are adapted to medical scans they can act as a tireless second reader: they process an image in a fraction of a second, they are consistent from one case to the next, and they can point to the region of the image that influenced their answer. This project explores that idea for one concrete problem — classifying brain MRI slices into glioma, meningioma, pituitary tumour or no tumour — and wraps the result in a web application that anyone can use to upload a scan and see an explained prediction.

1.2 Brain Tumours and Their Types

A brain tumour is an abnormal growth of cells inside the skull. Tumours that begin in the brain are called primary tumours, while those that spread from another part of the body are called secondary or metastatic tumours. A primary tumour may be benign, growing slowly

and pressing against surrounding tissue, or malignant, invading neighbouring tissue and tending to return after treatment. Some tumours cause no symptoms for a long time; others produce headaches, seizures, vision problems or changes in behaviour as soon as they disturb an active region of the brain. The World Health Organization classification of central nervous system tumours recognises well over a hundred distinct tumour types [17], but three groups account for the majority of the cases that appear in public classification datasets, and they are the focus of this work.

- **Glioma** — tumours that arise from glial cells, the supportive cells that surround and protect neurons. Gliomas are commonly reported to make up about one-third of all brain tumours. Their prognosis varies widely with grade, and their boundaries are often diffuse, which makes them visually subtle on MRI.
- **Meningioma** — tumours that grow from the meninges, the protective membranes that cover the brain and spinal cord. They account for roughly 30% of primary brain tumours. Most are benign and well-defined, although some behave more aggressively.
- **Pituitary tumour (adenoma)** — tumours of the pituitary gland at the base of the skull. Most are benign, but because the gland regulates hormones, even a small tumour can cause endocrine disturbances and, by pressing on the optic nerves, visual loss.
- **No tumour** — scans of healthy brains, which the classifier must be able to separate from the three tumour groups to avoid false alarms.

1.3 Role of MRI in Brain Tumour Detection

Magnetic Resonance Imaging is the preferred technique for examining brain tumours. It uses a strong magnetic field and radio-frequency pulses to build detailed pictures of soft tissue without exposing the patient to ionising radiation. Different pulse sequences (T1-weighted, T2-weighted, FLAIR and contrast-enhanced T1) highlight different tissue properties, and together they let a radiologist separate normal tissue, tumour, swelling and cysts. Compared with CT, MRI offers far better soft-tissue contrast, which is exactly what is needed to see the edge of a tumour or a small lesion near the skull base.

A single MRI study consists of a stack of two-dimensional slices taken in the axial, coronal or sagittal plane. The images used in this project are individual two-dimensional slices, which keeps the problem tractable on ordinary hardware and matches the format of the most widely

used public datasets. Because a slice shows only a thin section of the head, the same tumour can look very different depending on the plane and level at which it is cut — one of the reasons why automatic classification is a non-trivial task.

1.4 Limitations of Manual Diagnosis

Reading an MRI scan is a skilled and time-consuming task, and the traditional workflow has several well-known weaknesses.

- **Workload and delay.** Radiology departments handle large and growing numbers of studies. Each case may wait in a queue, which delays diagnosis for patients who may be seriously ill.
- **Subjectivity.** Two readers can disagree on the same image, and the same reader can reach different conclusions on different days. Fatigue is a documented source of missed findings late in a long reporting session.
- **Visual similarity between classes.** Gliomas and meningiomas can look alike on certain slices, and a small pituitary lesion can be mistaken for normal anatomy. Careful comparison across sequences is required, and it is not always available.
- **Limited access to experts.** In smaller hospitals and remote regions a neuro-radiologist may not be available, so scans are read by generalists or sent elsewhere, adding further delay.
- **Opacity of automated tools.** Where software assistance exists, it often returns only a label, with no indication of why, which makes it hard for a clinician to decide how much to trust it.

1.5 Role of Artificial Intelligence in Medical Imaging

Artificial intelligence, and deep learning in particular, has produced some of the most visible advances in medical image analysis. Instead of relying on hand-crafted measurements such as texture statistics or shape descriptors, a deep network learns the relevant features directly from examples. In the lower layers it discovers edges and intensity changes; in the middle layers, textures and local structures; and in the deeper layers, whole-object patterns that correlate with the clinical label.

For brain tumours, AI is typically applied to three tasks: **detection** (is there an abnormality?), **classification** (which type of tumour is it?) and **segmentation** (which pixels belong to the

tumour?). This project addresses the first two together as a four-class classification problem, and adds a fourth capability that is increasingly considered essential for clinical acceptance — **explainability**. A heatmap produced by the Grad-CAM technique [3] shows which part of the scan the network relied on, so that a user can check whether the model is looking at the lesion or at something irrelevant.

1.6 Convolutional Neural Networks and Transfer Learning

A convolutional neural network is built from layers that slide small learnable filters across an image. Early filters respond to simple patterns such as edges, and later layers combine them into increasingly abstract representations. Pooling layers shrink the spatial resolution so that deeper layers see a wider part of the image, and a final set of fully connected layers maps the learned features to class probabilities through a softmax function.

Training a deep CNN from scratch normally needs hundreds of thousands of labelled images, but medical datasets are small because annotation requires expert time. **Transfer learning** addresses this: a network is first trained on a very large general-purpose collection such as ImageNet [4], and its weights are then reused as the starting point for the medical task. The general visual building blocks learned from natural photographs — edges, corners, textures — transfer surprisingly well to MRI. In this project two such backbones are used: VGG16 [1], as a transparent baseline, and EfficientNet-B0 [2], a modern and much more parameter-efficient architecture.

1.7 Motivation of the Project

Several observations motivated this work. First, public brain-MRI datasets are now large enough to train useful classifiers on a single workstation, so a student project can reach a meaningful level of accuracy without specialised infrastructure. Second, many published studies stop at a notebook or a reported accuracy figure; very few turn the model into something a non-programmer can operate. A working web application forces the engineering questions — how the model is loaded, how images are validated, how errors are reported, how concurrent users are handled — to be answered rather than ignored.

Third, a classifier that outputs only a label is hard to trust. Adding a visual explanation changes the interaction: the user can see whether the highlighted region coincides with the visible lesion, which builds appropriate confidence in correct predictions and exposes suspicious ones. Finally, the project offered a realistic lesson in model quality. A model

inherited from earlier experiments reported 80.8% accuracy overall, yet detected only 18% of gliomas — a failure that an overall accuracy figure hides. Diagnosing and repairing that failure became a central part of the work.

1.8 Problem Statement

Manual interpretation of brain MRI scans is slow, subjective and dependent on scarce expertise, while existing automated tools are either research notebooks that cannot be used by a clinician or black boxes that return a label without justification. The problem addressed in this project is therefore to **design, train and deploy an accurate and explainable system that classifies a brain MRI slice as glioma, meningioma, pituitary tumour or no tumour, presents the result together with per-class probabilities and a visual attention map through a simple web interface, and remains reliable under invalid input and concurrent use.**

1.9 Objectives of the Project

The project was organised around the following objectives.

1. To study the types of brain tumours, the role of MRI and the state of the art in deep-learning-based tumour classification.
2. To prepare the Kaggle Brain Tumor MRI dataset (5,712 training and 1,311 test images, four classes) with a consistent pre-processing pipeline that removes the black scan background and standardises the input size and intensity range.
3. To build and compare two transfer-learning classifiers — a two-stage VGG16 baseline and an EfficientNet-B0 network — and to select the best one using a validation set, keeping the test set untouched until the final evaluation.
4. To diagnose the weak glioma performance of the starting EfficientNet checkpoint and to improve it through fine-tuning, targeting at least 95% overall accuracy and at least 90% recall for every class.
5. To integrate Grad-CAM so that each prediction is accompanied by a heatmap of the regions that influenced it.
6. To develop a Flask REST API and a React web interface that expose the model, with input validation, clear error messages and thread-safe inference.

7. To verify the complete system through automated tests, a full test-set run through the live API and a latency benchmark.

1.10 Scope of the Project

The scope of the project is deliberately bounded so that every claim in this report can be backed by an experiment that was actually run.

Included in Scope

- Four-class classification of two-dimensional brain MRI slices: glioma, meningioma, pituitary tumour and no tumour.
- Transfer learning with VGG16 and EfficientNet-B0, including fine-tuning, class weighting, label smoothing and data augmentation.
- Grad-CAM heatmap generation for every prediction, returned alongside the diagnosis and the probability of each class.
- A Flask API with health, model-information and prediction endpoints, and a React single-page interface with image upload, result display, a Grad-CAM view toggle and a session history.
- Offline evaluation (accuracy, precision, recall, F1, confusion matrices), automated backend tests, full API evaluation and latency measurement.

Excluded from Scope

- Clinical use. The system is an academic prototype and is not a medical device; it has not been validated on patient data from a hospital.
- Tumour segmentation, grading, volumetry or any three-dimensional analysis of full MRI studies.
- Integration with hospital systems such as PACS, DICOM routers or electronic health records.
- User accounts, persistent storage of patient data and compliance work for regulations such as HIPAA or GDPR.
- Detection of non-MRI or out-of-distribution images; the model assumes that the uploaded image is a brain MRI slice.

1.11 Key Features of the Proposed System

- **Four-class diagnosis** with a confidence score and a probability for every class.
- **Explainable output**: a Grad-CAM heatmap overlay that can be toggled against the original scan.
- **Fast inference**: the median end-to-end request, including heatmap generation, takes about 150 ms on a CPU-only laptop.
- **Two interchangeable model back-ends** (EfficientNet-B0 and VGG16) behind a single API contract, selected by an environment variable.
- **Robust input handling**: unsupported types, corrupt files and oversized uploads are rejected with clear JSON error messages.
- **Thread-safe serving**: inference is serialised so that concurrent users cannot corrupt the model's internal state.
- **Reproducible evaluation**: scripts and tests that regenerate every metric reported in this document.
- **Modern interface**: a responsive React application with an animated three-dimensional hero, drag-and-drop upload and a session history.

1.12 Expected Outcomes

At the start of the project the following outcomes were expected, and Chapter 8 reports how each of them was met.

- A trained classifier with at least 95% overall accuracy and no class with recall below 90% on the held-out test set.
- A working end-to-end web application in which an uploaded scan produces a diagnosis and heatmap in well under a second.
- Evidence that the live API reproduces the offline evaluation exactly, so that no accuracy is lost in the serving path.
- A documented comparison of the models tried, including the reasons why the earlier checkpoint failed on gliomas.
- A clear statement of the system's limitations so that the results are not over-interpreted.

1.13 Organisation of the Report

The remainder of this report is organised as follows. **Chapter 2** reviews related work on deep-learning-based brain tumour classification, transfer-learning architectures and explainability. **Chapter 3** states the functional and non-functional requirements. **Chapter 4** describes the system architecture and the UML and data-flow diagrams. **Chapter 5** explains the methodology, dataset and training strategy. **Chapter 6** presents the implementation with code listings and screenshots of the application. **Chapter 7** reports the testing and results, and **Chapter 8** concludes with the achievements, limitations and directions for future work.

Chapter 2
Literature Survey

2.1 Introduction to Existing Systems

Automatic analysis of brain MRI has been studied for more than two decades. Early systems relied on classical image processing: filtering to suppress noise, thresholding or clustering to isolate suspicious regions, and hand-designed texture or shape features fed to conventional classifiers such as support vector machines. These pipelines work on carefully selected data but are fragile, because every new scanner, sequence or patient population changes the statistics that the hand-crafted features depend on.

Since 2012 the field has moved decisively towards deep learning. Convolutional networks learn their own features, adapt to new data by re-training, and routinely reach accuracies above 95% on public four-class datasets. This chapter surveys ten representative studies, summarises the network architectures on which this project builds, reviews the explainability technique used for the heatmaps, and closes with a comparison and a statement of the research gaps that the project addresses.

2.2 Study of Deep-Learning Approaches for Brain Tumour Detection

2.2.1 Segmentation and Edge-Based Methods

One line of work detects tumours through edge and region analysis. A study titled *A Superior Achievement of Brain Tumor Detection Using Segmentation Based on F-Transform* applies the fuzzy-transform (F-transform) to reduce noise and extraneous detail in MRI scans, and then uses morphological operations to improve contrast. Its main idea is to exploit the approximate left–right symmetry of a healthy brain: an asymmetric region is a candidate tumour. The work also discusses CNN designs such as VGG and ResNet as a way of moving from detection to classification. Such methods are interpretable and light-weight, but they depend on image alignment and struggle when a tumour lies close to the mid-line, where it does not break the symmetry.

2.2.2 Autoencoder-Based Approaches

A second family uses autoencoders, which are unsupervised networks that compress an image through an encoder and reconstruct it through a decoder. *Deep Learning Approach for Brain Tumor Detection and Segmentation* uses an autoencoder to retain important image characteristics while discarding redundant information; the reconstructed images are

normalised and segmented with K-means clustering, with dropout against over-fitting and the Adam optimiser [8] for training. *Brain Tumor Detection and Classification Using Deep Wavelet Auto-Encoder* first enhances MRI images with high-pass and median filtering, locates the tumour with seed-growing segmentation and then passes the result to a wavelet autoencoder for feature extraction. These methods reduce the need for labels, but their multi-stage pipelines are harder to deploy and tune than a single end-to-end classifier.

2.2.3 CNN Classification Networks

Most recent studies treat the task as image classification with a CNN. *Brain Tumor Detection from MRI Images Based on ResNet18* trains an 18-layer residual network [5] with stochastic gradient descent with momentum and reports better accuracy than CapsNet and GoogLeNet, crediting the residual blocks that ease the training of deep networks. *Multi-Classification of Brain Tumor Images Using Deep Neural Network* builds a 16-layer CNN with several convolutional layers, dropout against over-fitting and fully connected layers with a softmax output to separate several tumour classes. *Brain Tumor Detection Using Deep Learning Techniques* adopts MobileNet [6], whose depth-wise separable convolutions greatly reduce the number of parameters, together with global average pooling and the Adam optimiser. *Brain Tumor and Glioma Grade Classification Using Gaussian CNN* applies Gaussian filtering as pre-processing to a 16-layer network with normalisation, down-sampling and dropout components, and *Brain Tumor Detection in MRI Images Using Image Processing Techniques* describes a CNN with convolution, activation, max-pooling and fully connected layers trained with Adam, SGDM and RMSProp. Finally, *Non-invasive Metabolic Imaging of Brain Tumors in Precision Medicine* uses a sequential network of convolution layers with ReLU, pooling, a flatten layer and dense layers with softmax to output class probabilities.

2.2.4 Detection-Based Approaches

Detection and Classification of Brain Tumor Using Deep Convolutional Network combines a VGG-16 feature extractor with a Region-based CNN. A Region Proposal Network suggests candidate tumour locations and the system then selects the best bounding boxes for classification, with training by stochastic gradient descent. Detection networks localise a tumour, which classifiers cannot, but they need box annotations that most public classification datasets do not contain.

2.3 Overview of Transfer-Learning Architectures

2.3.1 VGG16

VGG16 [1] stacks thirteen 3×3 convolutional layers in five blocks, each followed by max-pooling, and ends with fully connected layers. Its design is uniform and easy to understand, which makes it a popular teaching and baseline model, and its ImageNet weights are widely available. The price is size: the convolutional base alone has about 14.7 million parameters, and with the dense classifier used in this project the model has about 21.1 million parameters and requires a full forward pass through large feature maps, which is slow on a CPU. In this project VGG16 serves as the baseline and as the legacy back-end of the API.

2.3.2 ResNet and MobileNet

ResNet [5] introduced *skip connections* that add a layer's input to its output, so that gradients can flow through very deep networks and training does not degrade as layers are added. MobileNet [6] replaces ordinary convolutions with depth-wise separable ones, cutting computation and parameters by roughly an order of magnitude at a small cost in accuracy. Both ideas reappear inside EfficientNet.

2.3.3 EfficientNet

EfficientNet [2] observes that depth, width and input resolution should be scaled together rather than independently. Its baseline, EfficientNet-B0, is found by neural architecture search and is built from inverted-residual (MBConv) blocks with squeeze-and-excitation attention. With only about 4.0 million parameters — roughly one-fifth of the VGG16 model used here — it matches or exceeds much larger networks on ImageNet. For this project that means a 16 MB checkpoint, fast CPU inference and enough capacity to be fine-tuned on a few thousand MRI slices.

2.4 Explainable AI and Grad-CAM

A deep network is often called a black box because its decision is distributed over millions of weights. In medicine this is a serious obstacle: a clinician must be able to judge whether a prediction is based on the lesion or on an artefact. Gradient-weighted Class Activation Mapping (Grad-CAM) [3] provides a simple answer. For a chosen class it computes the gradient of the class score with respect to the feature maps of the last convolutional layer, averages the gradients over each map to obtain an importance weight, forms a weighted sum of the maps and keeps only the positive part. The result is a coarse map that is up-sampled to

the image size and overlaid on the scan, with warm colours marking the regions that increased the score of the predicted class.

$$L(x, y) = ReLU(\Sigma_k \alpha_k \cdot A_k(x, y)), \quad \alpha_k = (1 / Z) \Sigma_i \Sigma_j \partial y^c / \partial A_k(i, j)$$

Grad-CAM needs no change to the network architecture and no re-training, which is why it can be attached to both back-ends of this system. It is, however, an approximate visualisation: the maps are low resolution and show where the model looked, not proof that the model reasoned correctly.

2.5 Public Datasets for Brain Tumour Research

Three kinds of public data are used in the literature. The **BraTS** benchmark [15] provides multi-sequence MRI volumes of gliomas with expert segmentations and is the standard for segmentation research. The **Medical Segmentation Decathlon** [16] includes a brain-tumour task with multi-modal scans and tumour masks. For slice-level classification, the dataset collected by Cheng et al. [14] contains contrast-enhanced T1 slices of gliomas, meningiomas and pituitary tumours, and the **Kaggle Brain Tumor MRI dataset** [13] combines such public collections with a fourth class of healthy scans into 7,023 images. The last one is used in this project because its four-class labelling matches the clinical question of the project and its fixed training/testing split makes results comparable.

2.6 Comparative Analysis of Existing Approaches

Table 2.1 compares the families of approaches reviewed above with the system proposed in this report. The comparison is qualitative: the studies use different datasets and metrics, so their reported accuracies are not directly comparable.

Table 2.1: Comparison of Existing Approaches

Approach	Technique	Key Features	Limitations (relative to this project)
F-Transform segmentation	Fuzzy transform + morphology	Noise reduction; symmetry-based detection	Detection only; depends on image alignment
Autoencoder + K-means	Autoencoder, clustering	Unsupervised feature compression; segmentation	Multi-stage pipeline; no class probabilities
Deep wavelet autoencoder	Filtering, seed growing, DWAE	Strong feature extraction after segmentation	Complex pipeline; harder to deploy
ResNet18 classifier	Residual CNN, SGDM	Residual blocks ease deep training	Model only; no explanation or web interface
16-layer CNN / Gaussian CNN	Custom deep CNN, dropout	Multi-class and glioma-grade output	Trained from scratch; needs more data

Approach	Technique	Key Features	Limitations (relative to this project)
MobileNet classifier	Depth-wise separable CNN	Few parameters; efficient	No heatmap; no deployed application
VGG-16 + R-CNN	Region proposal + classifier	Localises tumours with boxes	Needs box annotations; heavy to run
Proposed system	VGG16 and fine-tuned EfficientNet-B0 + Grad-CAM	4-class diagnosis, probabilities, heatmap, Flask API and React UI, tested end to end	2-D slices only; not clinically validated

2.7 Research Gaps Identified

The survey shows that classification accuracy on public datasets is no longer the main obstacle. The gaps that remain are mostly about turning models into dependable, understandable tools.

- **The deployment gap.** Most studies end with a trained model and an accuracy table. Few describe how the model is served, how invalid input is handled or how the service behaves under concurrent use. This project builds and tests the whole path from browser to model.
- **The explainability gap.** Many classifiers return a label only. Combining the prediction with Grad-CAM in the same response lets the user inspect every result.
- **The per-class reliability gap.** Overall accuracy can hide a collapsed class; the starting checkpoint of this project scored 80.8% overall but detected 18% of gliomas. Reporting per-class recall and selecting checkpoints on the worst class addresses it.
- **The efficiency gap.** Heavy architectures such as VGG16 take seconds per image on a CPU. A compact network such as EfficientNet-B0 allows interactive use without a GPU.
- **The verification gap.** Offline accuracy is rarely cross-checked against the deployed service. Here the full test set is run through the live API to confirm that the two agree.

2.8 Summary of Literature Survey

The literature shows a clear progression from hand-crafted segmentation pipelines to end-to-end CNN classifiers, and from large networks trained from scratch to compact pre-trained backbones that are fine-tuned on a small medical dataset. Grad-CAM offers a low-cost route to explainability. What is less often addressed is the engineering around the model —

serving, validation, robustness and per-class verification. These gaps define the contribution of this project, whose requirements are specified in the next chapter.

Chapter 3
Analysis / Software Requirements Specification (SRS)

3.1 Introduction

This chapter specifies what the Brain Tumor Detection system must do and the qualities it must have. The requirements were derived from the problem statement in Chapter 1, from the gaps identified in the literature survey and from the experience of earlier prototypes (a VGG16 training script and an EfficientNet checkpoint). Each functional requirement is written in the form "The system shall…" and is verified by at least one test in Chapter 7.

3.2 Purpose of the System

The purpose of the system is to give a user a fast, explained second opinion on a brain MRI slice. The user uploads an image through a web page; the system decides whether the slice shows a glioma, a meningioma, a pituitary tumour or no tumour, states how confident it is, and highlights the part of the image that drove the decision. The system supports learning, demonstration and research on explainable medical-image classification. It is explicitly **not** a diagnostic device and displays a disclaimer to that effect.

3.3 Intended Users

- **Students and researchers** who want to explore how a deep-learning classifier and its Grad-CAM explanation behave on real MRI slices.
- **Clinicians and radiology trainees** who may use the tool as a demonstration of computer-aided reading; they are the audience for whom the explanation is designed, although the system is not validated for clinical use.
- **ML engineers and administrators** who retrain or replace the model, run the evaluation scripts and operate the API.
- Patients are not direct users of the system.

3.4 Product Scope

The product is a client–server web application. The client is a React single-page interface; the server is a Flask API that loads a PyTorch (or optionally TensorFlow) model once at start-up and serves predictions. The product accepts a single two-dimensional MRI image per request

in PNG, JPEG or BMP format and returns a JSON document containing the predicted class, the confidence, the probability of every class, the original image and the heatmap overlay as base64-encoded PNG images. It does not store uploaded images, maintain user accounts or interact with hospital systems.

3.5 MRI Image Input Processing

The input subsystem accepts a file chosen by clicking the upload area or by drag and drop, shows a preview, and sends it to the API as multipart form data. On the server the file is checked for the permitted extensions, decoded and converted to three-channel RGB so that greyscale, palette and transparent images are all handled uniformly. The image is then cropped to the bounding box of non-black pixels (removing the empty scanner background), resized to 224×224 pixels and normalised with the ImageNet mean and standard deviation. This is exactly the pre-processing used during training, because any difference between training and serving pre-processing would silently reduce accuracy.

3.6 Tumour Classification using Deep Learning

The classification subsystem passes the pre-processed tensor through the selected network and applies a softmax to the four output scores. The class with the highest probability is reported as the prediction and its probability as the confidence. The default network is the fine-tuned EfficientNet-B0; the VGG16 network remains available as an alternative back-end. The class order is read from a `class_indices.json` file so that index 0 always means glioma, 1 meningioma, 2 no tumour and 3 pituitary.

3.7 Explainability Mechanism (Grad-CAM)

For every prediction the system computes a Grad-CAM map from the last convolutional feature layer, resizes it to the region of the image that the network actually saw, and blends it over the original scan with a JET colour map at 40% opacity. The heatmap is zero outside the brain-region crop because that area was never given to the network. Both the original and the overlaid image are returned so that the interface can toggle between them.

3.8 Result Display System

The result panel shows a coloured diagnosis banner (red for a tumour class and green for no tumour) with the confidence and a one-line description of the predicted tumour type, an Original / Grad-CAM Heatmap toggle above the image, and a bar for each class with its

probability. A session history lists every scan analysed in the current browser session so that earlier results can be revisited.

Functional Requirements Summary

Table 3.1: Functional Requirements of the System

ID	Requirement	Priority
FR-01	The system shall accept an MRI image in PNG, JPEG or BMP format by click-to-browse or drag-and-drop and show a preview.	High
FR-02	The system shall reject a missing file, an unsupported type, a corrupt image and a file above 16 MB with a clear error message.	High
FR-03	The system shall crop the brain region, resize to 224×224 and normalise the image identically to training.	High
FR-04	The system shall classify the image as glioma, meningioma, pituitary tumour or no tumour.	High
FR-05	The system shall return the confidence and the probability of each of the four classes, summing to one.	High
FR-06	The system shall generate a Grad-CAM heatmap overlay for the predicted class.	High
FR-07	The system shall display a diagnosis banner stating either the possible tumour type or that no tumour was detected.	High
FR-08	The system shall let the user toggle between the original scan and the heatmap overlay.	Medium
FR-09	The system shall keep a session history of analysed scans and allow an earlier result to be reopened.	Medium
FR-10	The system shall expose a health endpoint that reports whether the model has been loaded.	Medium
FR-11	The system shall expose a model-information endpoint with the class list and saved evaluation metrics.	Medium
FR-12	The system shall allow the model back-end (EfficientNet or VGG16) to be selected by configuration.	Low
FR-13	The system shall display a notice that it is a research prototype and not a medical device.	High

3.9 Security and Privacy Requirements

- Uploaded images shall be processed in memory and shall not be written to disk or retained after the response is sent.
- The API shall limit the request size to 16 MB and shall reject non-image content before it reaches the model.
- Debug mode of the web server shall be disabled by default, and the server shall listen on localhost unless configured otherwise.
- Cross-origin access shall be controlled by the Flask-CORS extension; for any public deployment the permitted origins shall be restricted.

- The interface shall state that results are not a clinical diagnosis. Real patient data shall not be used without the consent and de-identification procedures required by regulations such as HIPAA or GDPR.

3.10 Performance Requirements

- The end-to-end latency of a prediction, including pre-processing, inference, Grad-CAM and response encoding, shall have a 95th percentile below one second for a single user on a CPU-only laptop.
- The service shall remain correct and responsive when several requests arrive at the same time.
- The system shall achieve at least 95% overall accuracy and at least 90% recall for every class on the held-out test set.
- The health endpoint shall respond in a few milliseconds without running the model.

3.11 Reliability and Availability

The server shall load the model once at start-up and report any loading problem through the health endpoint instead of failing silently. A malformed request shall never terminate the process: every error path returns a JSON error with an appropriate HTTP status. Because the Grad-CAM implementation stores intermediate tensors on a shared object, inference shall be serialised by a lock so that simultaneous requests cannot corrupt each other’s results.

3.12 Scalability

The prototype runs as a single process, which is sufficient for demonstration and small-group use. Scaling beyond that is possible without changing the API contract: several worker processes can be started behind a reverse proxy, each holding its own copy of the 16 MB model, or the model can be moved to a GPU. The stateless request design and the absence of server-side sessions make horizontal scaling straightforward.

Non-Functional Requirements Summary

Table 3.2: Non-Functional Requirements of the System

ID	Category	Requirement	Target
NFR-01	Accuracy	Overall accuracy on the 1,311-image test set	≥ 95%
NFR-02	Accuracy	Recall of every individual class	≥ 90%
NFR-03	Performance	P95 single-user latency of /api/predict	< 1 s
NFR-04	Performance	Concurrent users handled correctly	4 users, 0 errors

ID	Category	Requirement	Target
NFR-05	Reliability	Invalid input never crashes the server	400 / 413 JSON error
NFR-06	Consistency	Live API accuracy equals offline evaluation	Identical
NFR-07	Security	No image retained; debug off by default	Verified
NFR-08	Usability	Result readable at a glance with banner, bars and heatmap	Single screen
NFR-09	Portability	Runs on Linux, Windows and macOS with Python 3.10+ and a modern browser	CPU only
NFR-10	Maintainability	Back-ends swappable by configuration; tests and scripts regenerate all results	Provided

3.13 Hardware Requirements

Inference runs comfortably on an ordinary laptop. Training of the fine-tuned model was also carried out on the same CPU-only machine; a GPU would shorten training but is not required for any part of the system. Table 3.3 lists the machine on which all experiments in this report were run together with the recommended minimum for users who only want to run the application.

3.14 Software Requirements

Table 3.3: Hardware and Software Requirements

Item	Used for this project	Recommended minimum
Processor	Intel Core i7-12650H, 10 cores / 16 threads	4-core x86-64 CPU
Memory	31 GB RAM	8 GB RAM
Graphics	None used (CPU-only PyTorch)	Optional
Storage	About 600 MB for code, dataset and models	2 GB free
Operating system	Arch Linux (Linux 7.x)	Linux, Windows 10/11 or macOS
Python	3.14 with PyTorch 2.14, torchvision 0.29	Python 3.10+
Back-end libraries	Flask 3.1, Flask-CORS 6, Pillow 12, OpenCV, NumPy 2.5, scikit-learn 1.9	As listed in requirements.txt
Front-end	Node.js 26, React 19, Vite 8, Framer Motion, three.js	Node.js 18+
Testing tools	pytest 9, requests, Playwright + Chromium (screenshots)	pytest
Browser	Chromium-based browser with WebGL	Any current browser

3.15 User Interface Requirements

- A single page with a hero section, an upload card, a result card and a session-history card, readable on desktop and laptop screens.
- A dark theme with high-contrast text; ARIA labels on the upload area, the view toggle and the probability bars.

- Immediate feedback: the Analyze button shows a loading state, the preview appears as soon as a file is chosen, and errors appear in red below the button.
- A footer that shows whether the backend is reachable and the model's test-set accuracy.
- A visible disclaimer that the application is a research prototype and not a medical device.

3.16 Constraints and Assumptions

- The uploaded image is assumed to be a two-dimensional brain MRI slice; the system does not verify this, and a non-MRI image will still receive a label.
- The model was trained on one public dataset; accuracy on scans from other scanners or hospitals has not been measured.
- Training and evaluation were performed on a CPU, which limited the number of experiments that could be run.
- The system is limited to the four classes in the training data; other tumour types are mapped to the nearest known class.
- The browser must support WebGL for the animated hero; the analysis function works without it.

3.17 References Used in Analysis

- Louis et al., *The 2021 WHO Classification of Tumors of the Central Nervous System* [17] — tumour types and terminology.
- Nickparvar, *Brain Tumor MRI Dataset* [13] and Cheng et al. [14] — dataset definition and class labels.
- Selvaraju et al., *Grad-CAM* [3] — explainability requirement.
- Tan and Le, *EfficientNet* [2] and Simonyan and Zisserman, *VGG* [1] — model options.
- Flask [21] and React [22] documentation — interface and API conventions.

Chapter 4
System Design

4.1 Introduction

This chapter describes how the requirements of Chapter 3 are realised. It presents the overall architecture, the UML diagrams (use case, class and sequence), the data flow, and the design of each tier: the React front end, the Flask back end, the deep-learning model, the Grad-CAM explainer and the pre-processing pipeline. Security, performance and integration decisions are described at the end.

4.2 Overall System Architecture

The system follows a three-tier client–server architecture, shown in Fig 4.1. The **presentation tier** is a React application that runs in the browser. It never touches the model: it only calls the REST API with `fetch` and renders what comes back. The **application tier** is a Flask service that validates requests, pre-processes images, calls the model and assembles the JSON response. The **model tier** holds the PyTorch EfficientNet-B0 network and its Grad-CAM hooks, the optional TensorFlow VGG16 network, and the checkpoint and metadata files on disk. The model is loaded once, when the server starts, and kept in memory.

Separating the tiers mirrors how machine-learning products are normally built. Training, serving and the user interface are independent concerns: the model can be retrained without touching the website, the website can be redesigned without retraining, and the front end could be replaced by a mobile application without changing the API.

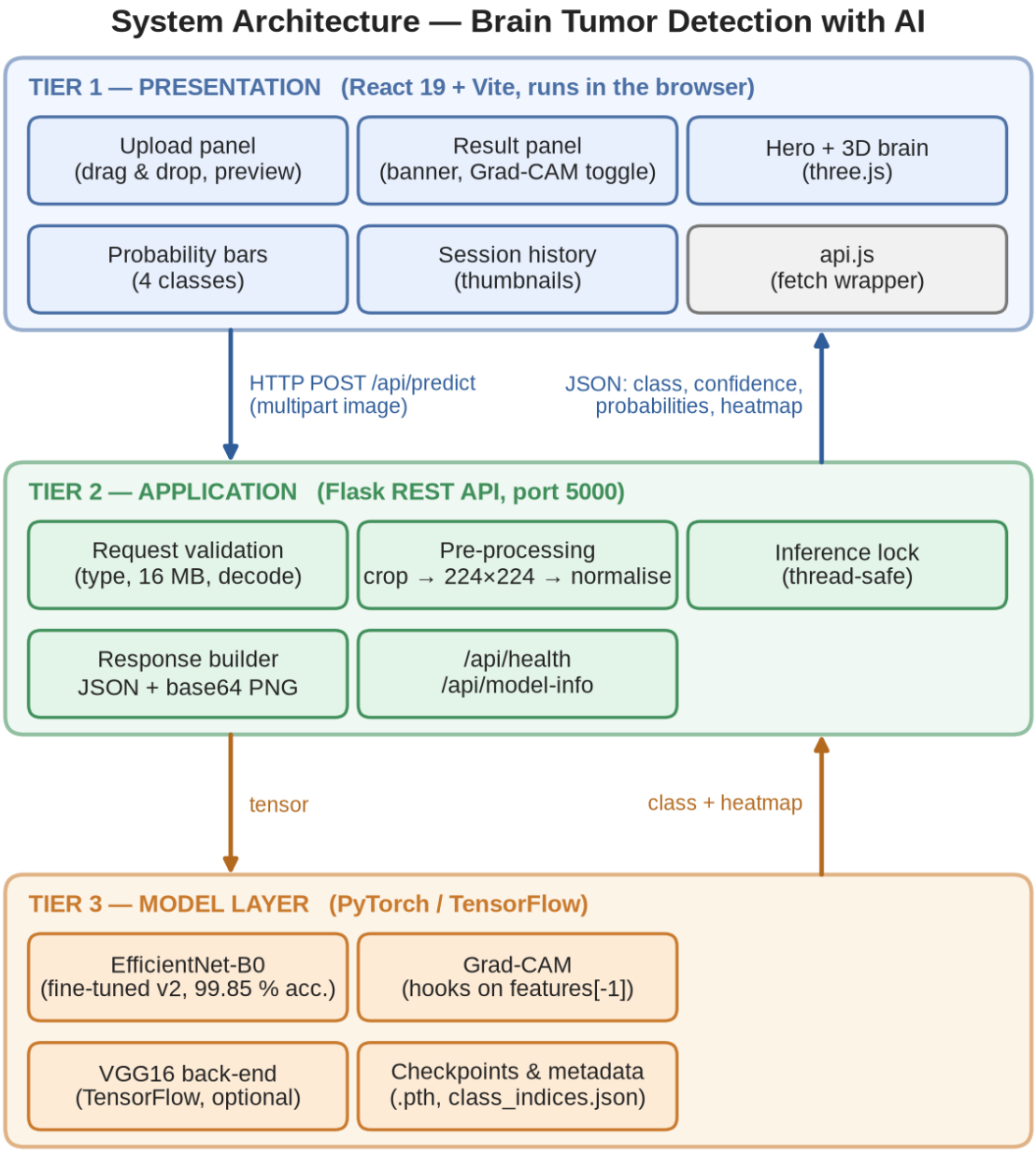


Fig 4.1: System Architecture — Brain Tumor Detection with AI

A request travels as follows. The browser posts the image to `/api/predict`. Flask validates it, converts it to RGB, crops and normalises it and runs one forward pass of the network under an inference lock. The Grad-CAM backward pass uses the activations and gradients captured by hooks on the last convolutional layer. The response builder then overlays the heatmap, encodes both images as PNG data URIs and returns a single JSON document, which the front end renders as the diagnosis banner, the image viewer and the probability bars.

4.3 Use Case Diagram

Fig 4.2 shows the two actors and their use cases. The **User** (a clinician, student or researcher) uploads a scan, triggers the analysis, reads the diagnosis and confidence, views and toggles the Grad-CAM heatmap, browses the session history and can check the health of the backend and the model's published metrics. The **ML Engineer / Administrator** additionally fine-tunes the model, evaluates it on the test set and switches between the EfficientNet and VGG16 back-ends. Analysing a scan always includes uploading it first, which is shown as an include relationship.

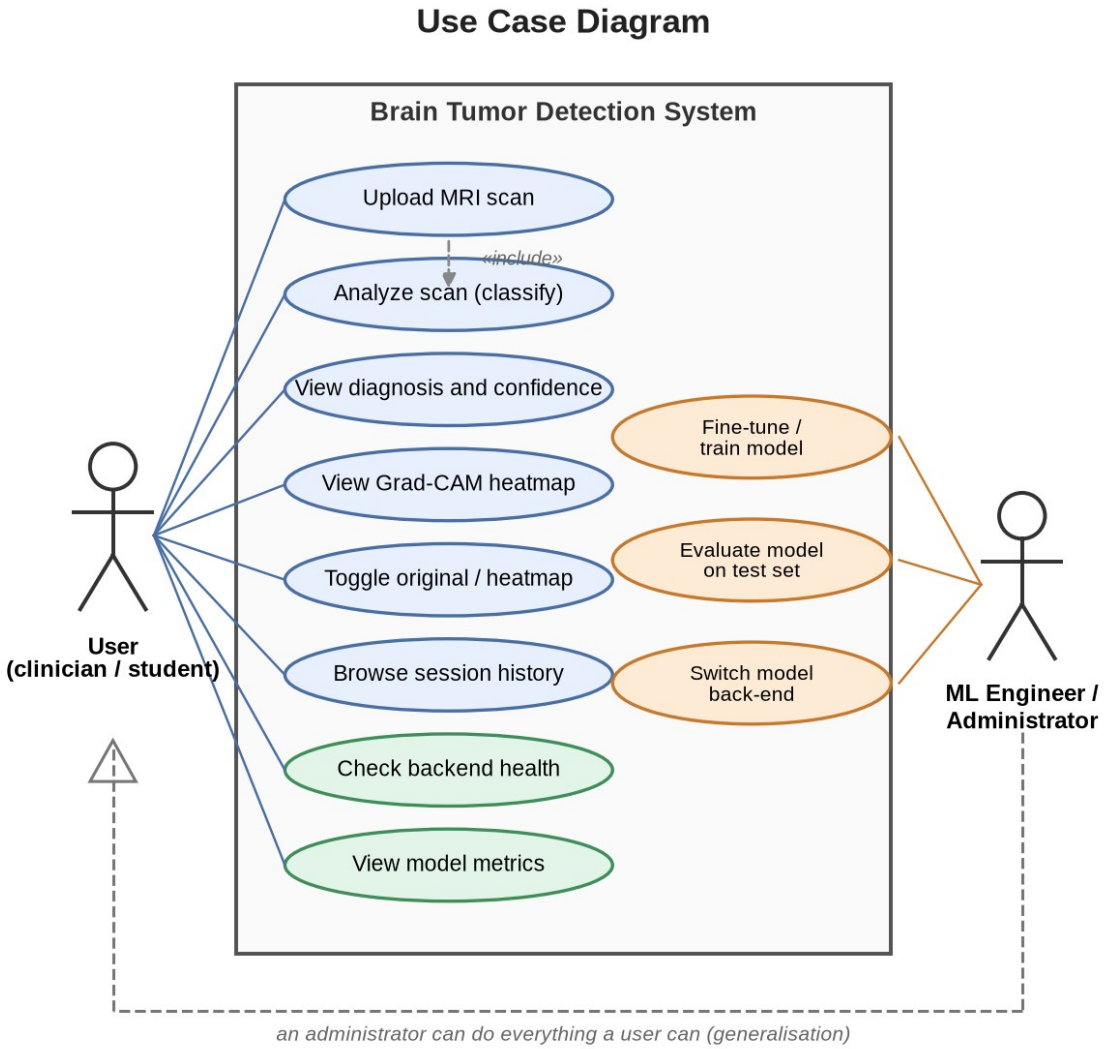


Fig 4.2: Use Case Diagram

4.4 Class Diagram

Fig 4.3 shows the principal classes and modules. On the front end, `App` owns the application state (the history list, the active result, the loading flag, the error text and the model

information) and contains the `UploadPanel`, `ResultPanel` and `HistoryPanel` components; `ResultPanel` in turn renders the `ProbabilityBars`. On the back end, the `FlaskApp` module holds the shared state and the inference lock and exposes the three routes. It uses the `Preprocessor` helpers, owns the `EfficientNetB0` model and calls the `GradCAM` class, which registers forward and backward hooks on the model's last feature layer.

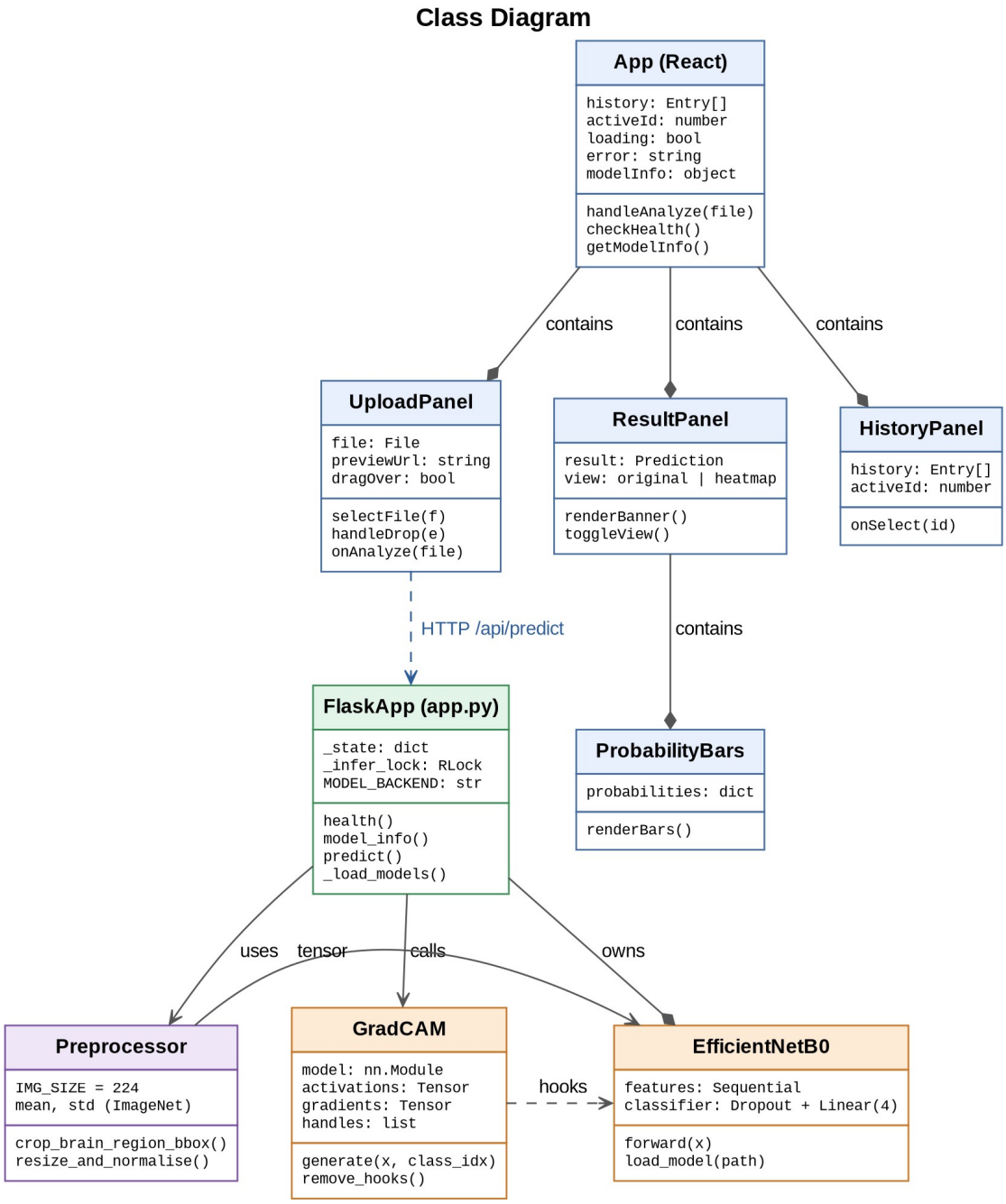


Fig 4.3: Class Diagram

4.5 Sequence Diagram

Fig 4.4 follows one analysis from the user's click to the rendered result. Steps 3 to 5 are the validation and pre-processing performed by the API; steps 6 to 8 are the forward and backward passes through the network; step 10 produces the heatmap overlay and encodes the images. The front end receives a single response and needs no further calls, which keeps the interaction simple and the latency low.

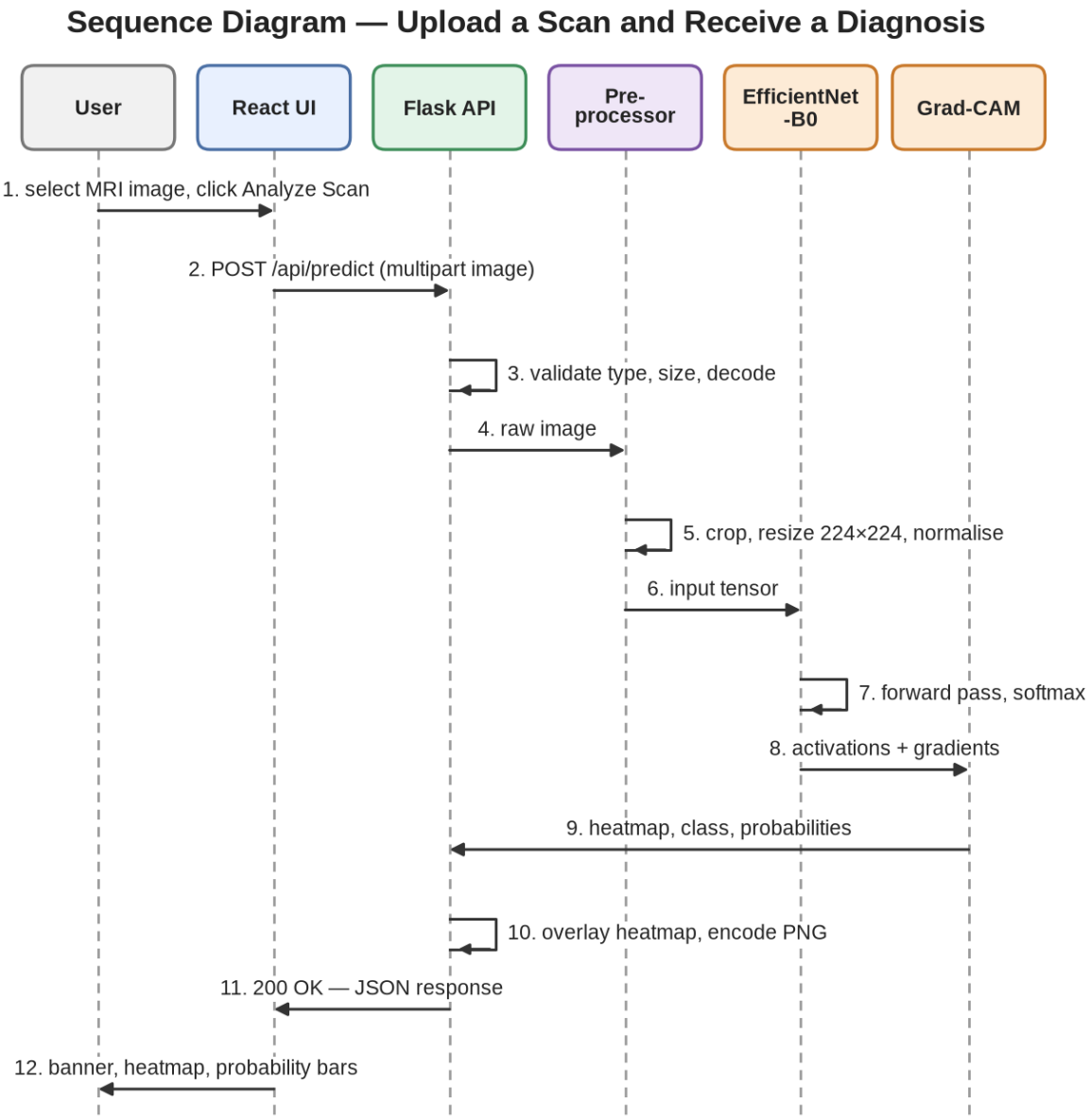


Fig 4.4: Sequence Diagram — Upload a Scan and Receive a Diagnosis

4.6 Data Flow of the System

The level-1 data flow diagram in Fig 4.5 decomposes the system into six processes. An uploaded image is validated (1.0), pre-processed into a tensor (2.0) and classified (3.0) using the weights stored in the model checkpoint (D1). Grad-CAM (4.0) uses the activations and gradients from the same pass. The response builder (5.0) combines the heatmap and probabilities with the class names and metrics read from D2, and the interface (6.0) renders the result and saves it in the session history (D3), which lives only in the browser's memory.

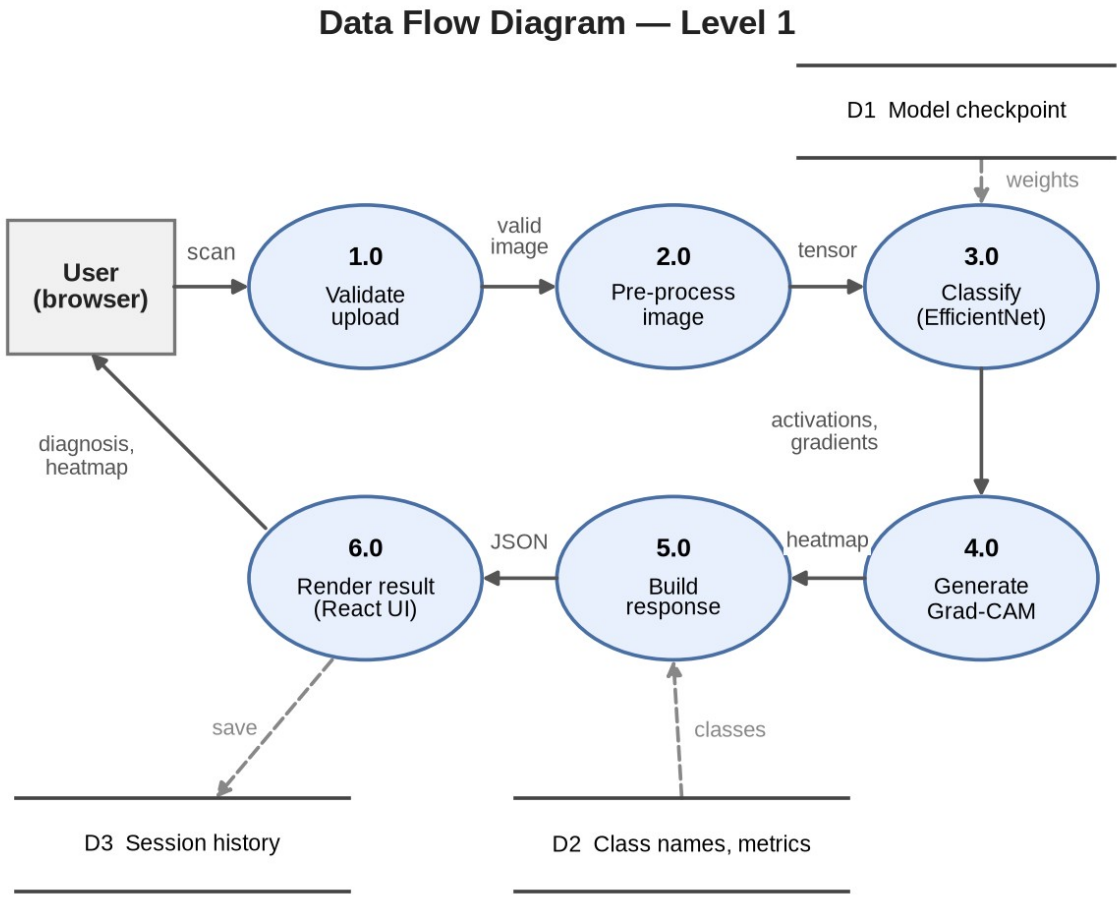


Fig 4.5: Data Flow Diagram — Level 1

4.7 Frontend Design (React UI Structure)

The user interface is a single-page React 19 application built with Vite. Its components map directly onto the use cases.

- **Hero** shows the title, the headline test-set accuracy read from `/api/model-info`, and an animated three-dimensional brain built with three.js and React Three Fiber.

- **UploadPanel** implements the drop zone, validates the file type on the client, shows a preview and calls the analyse handler.
- **ResultPanel** renders the diagnosis banner, the Original / Grad-CAM Heatmap toggle and the probability bars.
- **HistoryPanel** lists earlier results as thumbnails; clicking one makes it the active result.
- **api.js** wraps `fetch` for the three endpoints and turns every failure into a readable error message.

State is kept in the top-level `App` component with React hooks; there is no global store because the data model is tiny. Animations use Framer Motion and are limited to transitions between results. In development the Vite server proxies `/api` to Flask, so the browser sees a single origin and no CORS configuration is needed.

4.8 Backend Design (Flask Architecture)

The back end is a single Flask module with three routes: `GET /api/health` (liveness and model status), `GET /api/model-info` (classes and saved metrics) and `POST /api/predict` (the analysis). A dictionary `_state` holds the loaded model, the Grad-CAM object and the class names. Loading is lazy and idempotent: the first request, or the explicit call at start-up, loads the model and warms it up with a dummy image so that the first real user does not pay for PyTorch's first-call overhead. Any loading error is stored and reported by the health endpoint and by the routes with a 503 status.

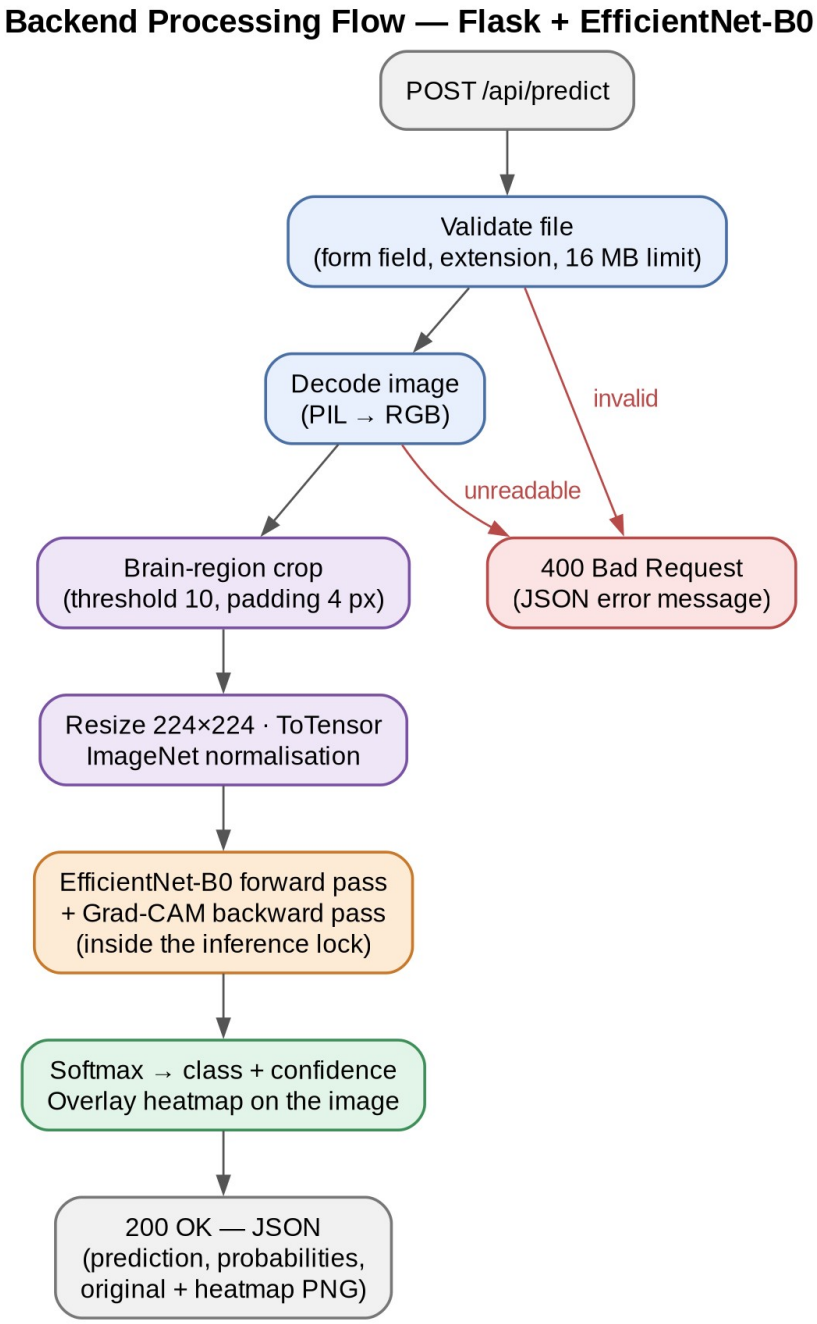


Fig 4.6: Backend Processing Flow — Flask + EfficientNet-B0

The model-selection logic is isolated in one function. By default the service uses the fine-tuned checkpoint in `model/efficientnet_v2/`; if it is missing it falls back to the original checkpoint folder and then to the first training run, and an environment variable can override all of these. This makes it safe to replace the model without editing code.

4.9 Model Design

Fig 4.7 shows the inference pipeline. After pre-processing, the image enters the EfficientNet-B0 backbone, which consists of seven stages of MBConv blocks and produces 1,280 feature maps of size 7×7. Global average pooling collapses each map to a single number, and a classifier head — dropout with probability 0.4 followed by a linear layer from 1,280 to 4 units — produces the class scores that the softmax converts to probabilities. The complete model has 4,012,672 parameters, of which only 5,124 belong to the new head; everything else is initialised from pre-trained weights and fine-tuned.

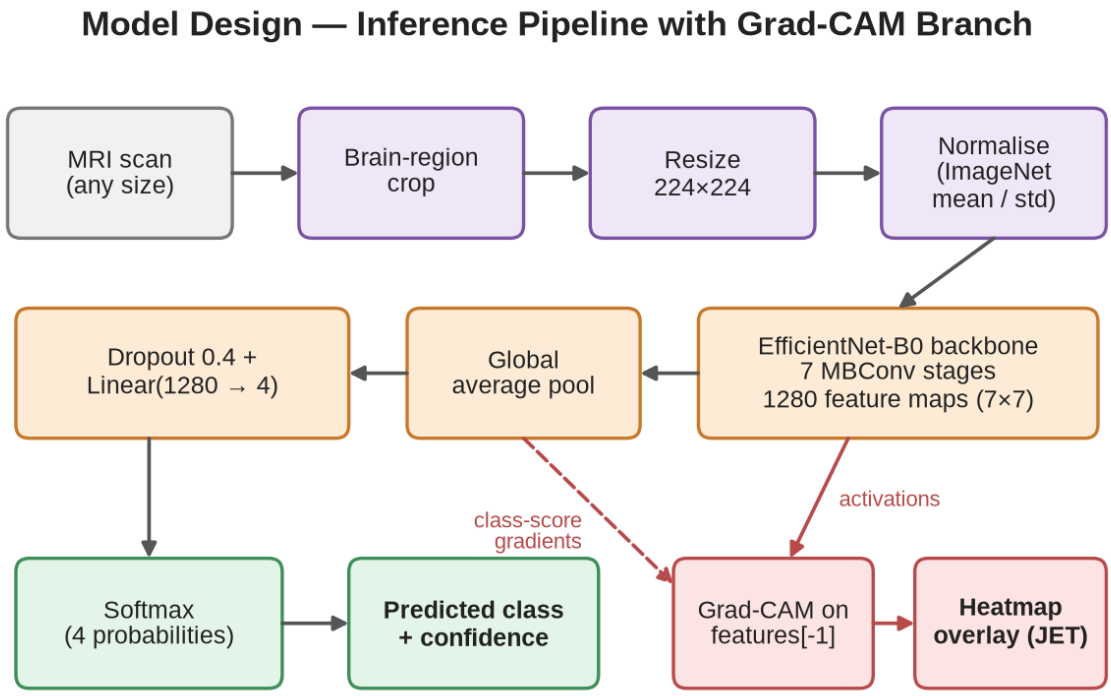


Fig 4.7: Model Design — Inference Pipeline with Grad-CAM Branch

The legacy VGG16 baseline uses two networks: a convolutional feature extractor (VGG16 without its top, 14.7 million parameters) and a classifier head of Flatten, Dense(256, ReLU), Dropout(0.5) and Dense(4, softmax). They are stored as two files because Grad-CAM needs direct access to the output of the last convolutional layer, which Keras hides inside a single nested model.

4.10 Grad-CAM Design

For the EfficientNet back-end the Grad-CAM class registers a forward hook on `model.features[-1]`, the final 1×1 convolution of the backbone, which retains a 7×7

spatial map of high-level features. The hook stores the activations; a backward hook stores the gradients of the predicted class score. The per-channel mean of the gradients gives the weights, the weighted sum of channels is passed through ReLU and scaled to the range 0 to 1. Because the network only saw the cropped brain region, the map is resized to that region and pasted into an all-zero canvas of the display image, so that the heatmap aligns with the original scan.

4.11 Image Pre-processing Design

Pre-processing has three steps, applied identically in training, offline evaluation and the API. First, a threshold of 10 on the grey-scale image marks the non-black pixels, and the bounding box of those pixels, padded by four pixels, is cropped; this removes the black border that surrounds most scans and gives the brain more of the 224×224 input. Second, the crop is resized to 224×224 pixels. Third, the pixel values are scaled to [0, 1] and normalised with the ImageNet mean (0.485, 0.456, 0.406) and standard deviation (0.229, 0.224, 0.225). The crop box is computed on the native-resolution image, not on a down-scaled display copy, to avoid a train/serve mismatch for large uploads.

4.12 Security Considerations

Security is addressed at the level appropriate for an academic prototype. Uploads are limited to 16 MB and checked for type and decodability before reaching the model; oversized requests receive a JSON 413 error. Images are held in memory only. Flask's debug mode, which would expose an interactive debugger, is off by default and the server binds to localhost unless configured otherwise. The model files are loaded with `torch.load` from a trusted local path; the loader never fetches weights from a user-controlled location. Authentication, rate limiting and restricted CORS origins would be required before any public deployment.

4.13 Performance Optimisation

- **Compact model.** EfficientNet-B0 has about one-fifth of the parameters of the VGG16 baseline and answers a request in about 0.15 s on a CPU, against roughly 2.7 s per image for the VGG16 back-end measured in the earlier API evaluation.
- **Load once, warm up once.** The model is loaded at start-up and exercised with a dummy image, so no request pays the initial cost.

- **Single pass for prediction and heatmap.** The forward pass, the gradient computation and the overlay are done in one request without re-running the network.
- **Cached training data.** During fine-tuning, images are cropped and resized once and kept in memory, so augmentation operates on small cached arrays instead of re-reading 512×512 JPEG files every epoch.
- **Serialised inference.** A lock prevents data races on shared hook state; with a single CPU model, concurrent requests are queued, and the measured 4-user throughput is 13 requests per second.

4.14 Integration Strategy

The tiers are integrated through a narrow, documented contract: three endpoints and one JSON schema. The response of `/api/predict` has the fields `prediction`, `tumor_detected`, `confidence`, `probabilities`, `original_image` and `heatmap_overlay`. Both model back-ends return exactly this structure, so the front end is unaware of which one is active. During development the Vite proxy forwards `/api` calls to Flask; in a deployment the same effect is obtained by a reverse proxy. The offline evaluation script and the live API evaluation script share the class order file, which guarantees that a label index means the same thing everywhere.

Chapter 5
Methodology

5.1 Development Methodology (Iterative / Agile Approach)

The project was developed iteratively. Instead of designing everything first and building it at the end, the team repeated a short cycle — study, build, test, review — and finished each cycle with something that worked end to end (Fig 5.1). The first cycle produced a working VGG16 model and training script. The second added a Flask API and a React interface around it. The third trained an EfficientNet-B0 model and made it the default. The fourth was triggered by a review of per-class results: the EfficientNet checkpoint detected only 18% of gliomas, so the cycle diagnosed the problem, fine-tuned the model and re-tested the whole system.

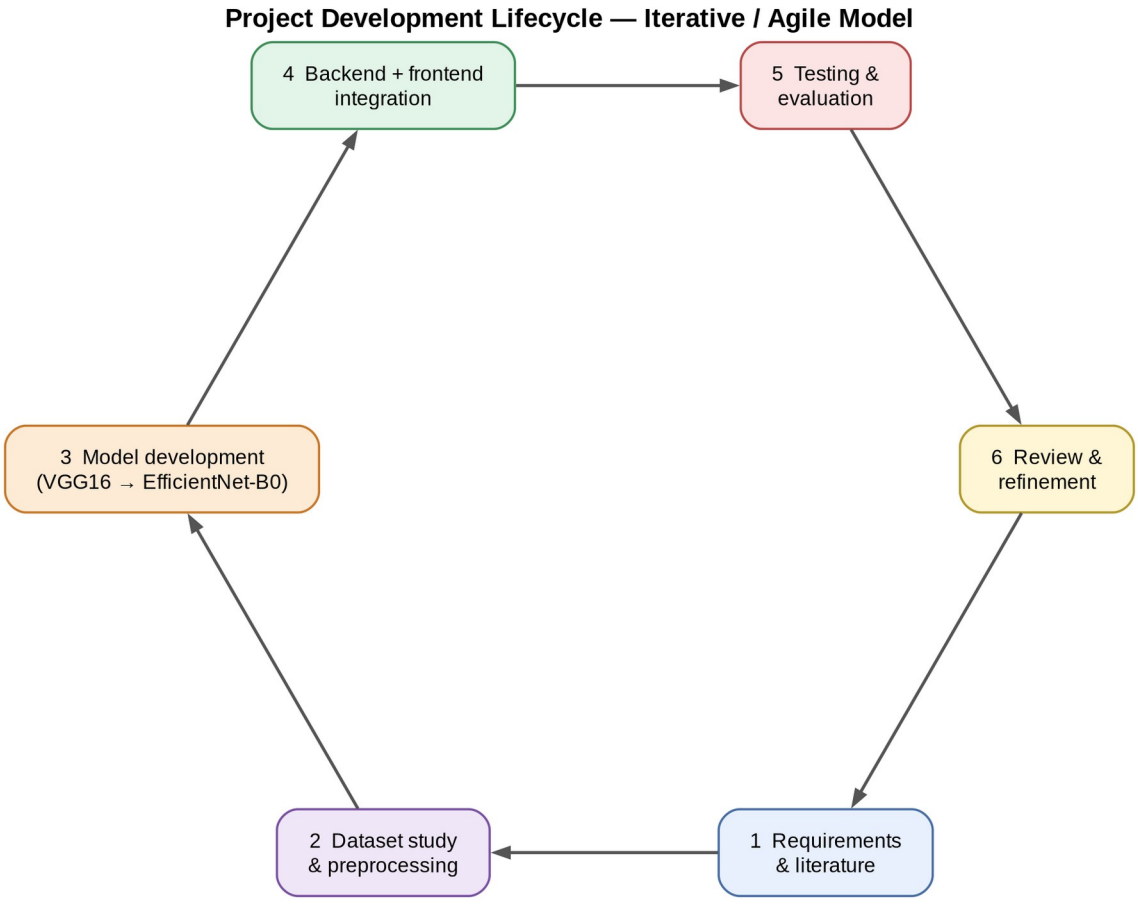


Fig 5.1: Project Development Lifecycle — Iterative / Agile Model

The iterative approach suited a project whose central risk — whether the model would be accurate enough — could only be measured by running experiments. Each review produced a

decision that fed the next cycle: replace the baseline, change the loss, fix the serving path, add a test.

5.2 Project Planning Strategy

Work was divided into the milestones shown in Table 5.1. Tasks that could run in parallel — for example the Flask API and the React interface — overlapped, and model experiments were scheduled early because they were the longest and least predictable activity. The Gantt chart in Fig 5.5 shows the planned overlap of these activities over fifteen weeks.

Table 5.1: Project Milestones and Timeline

Milestone	Deliverable	Planned weeks
M1 — Study	Literature survey and requirements specification	1–4
M2 — Data	Dataset study, class balance analysis and pre-processing pipeline	3–5
M3 — Baseline	Two-stage VGG16 classifier with Grad-CAM; 97.10% test accuracy	4–7
M4 — Second model	EfficientNet-B0 training script, checkpoint and evaluation	6–9
M5 — Integration	Flask API with dual back-end; React interface	8–12
M6 — Optimisation	Diagnosis of glioma failure; fine-tuned EfficientNet-B0 (99.85%)	11–13
M7 — Verification	Automated tests, full API evaluation, latency benchmark	12–14
M8 — Documentation	Project report and presentation	13–15

5.3 Requirement Gathering Process

Requirements were gathered from three sources. The problem statement defined the clinical task and the need for an explainable output. A review of the existing implementations — the earlier VGG16 training script and the EfficientNet training script — showed which engineering questions were still open (serving, input validation, concurrency). Finally, walking through the journey of a user who wants to check a scan produced usability requirements such as a preview before analysis, an Original / Heatmap toggle and a history of previous results. These were consolidated into the specification in Chapter 3.

5.4 Dataset Description and Preparation

The system uses the Kaggle Brain Tumor MRI dataset [13], which compiles images from several public sources, including the collection of Cheng et al. [14], into four classes. The folder structure follows the usual convention: `Training/` and `Testing/` directories with one sub-folder per class. Table 5.2 gives the composition, and Fig 5.2 and Fig 5.3 show the class distribution and a sample of images. The images vary in size (for example 512×512 for most

glioma and pituitary slices and roughly 200–250 pixels per side for many no-tumour slices) and in plane, which is why a size-independent pre-processing step is needed.

Table 5.2: Composition of the Dataset

Class	Training images	Testing images	Total
Glioma	1,321	300	1,621
Meningioma	1,339	306	1,645
No tumour	1,595	405	2,000
Pituitary	1,457	300	1,757
Total	**5,712**	**1,311**	**7,023**

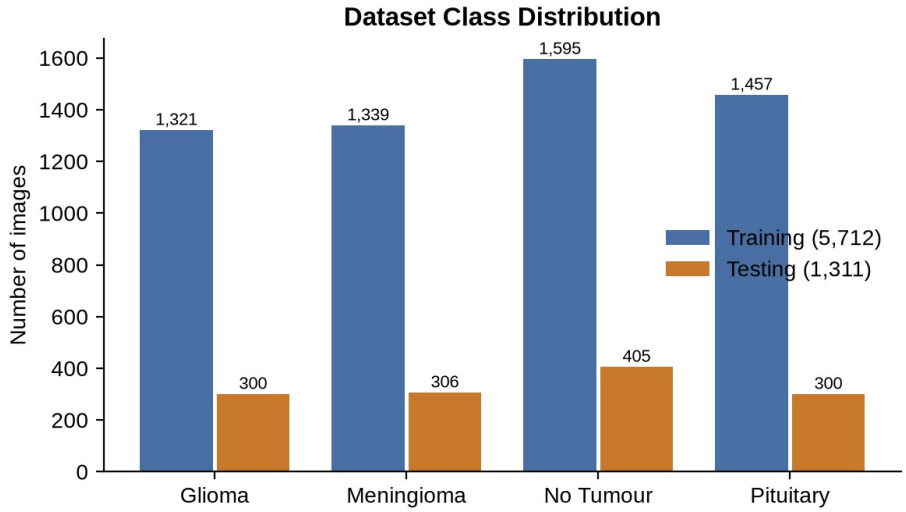


Fig 5.2: Dataset Class Distribution

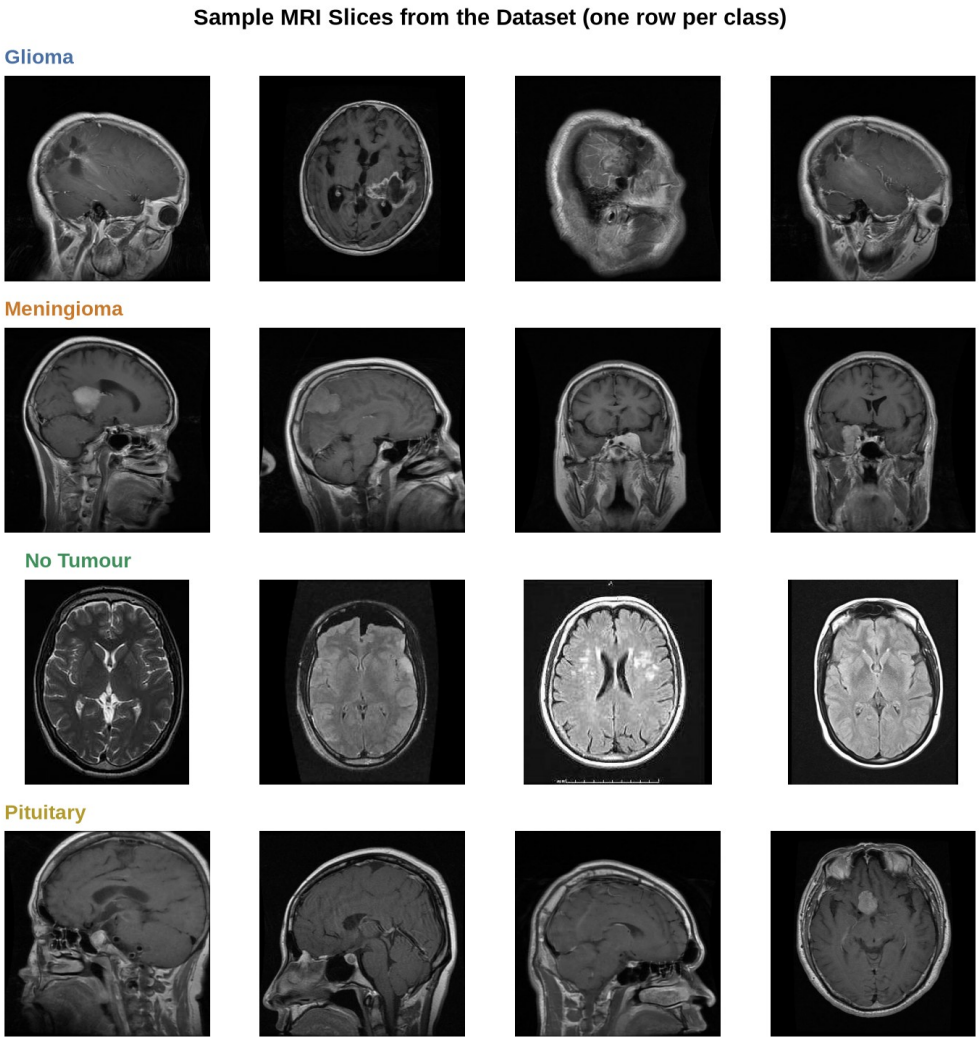


Fig 5.3: Sample MRI Slices from the Dataset (one row per class)

The classes are moderately balanced (the largest has about 1.2 times as many images as the smallest), so no re-sampling was needed; class weights in the loss function compensate for the remainder. A **validation set** was carved out of the training folder (15%, 856 images, fixed random seed 42) and used for checkpoint selection and early stopping. The **test set** was used only for the final evaluation and was never used to choose a model, a hyper-parameter or a checkpoint.

5.5 Data Pre-processing and Augmentation

All images, in training and in serving, pass through the pre-processing described in Chapter 4: brain-region crop, resize to 224×224 and ImageNet normalisation. During fine-tuning, the following random augmentations are applied to the training images only: horizontal flip, rotation up to ±15°, translation up to 6% of the image size, scaling between 88% and 112%, and brightness and contrast jitter of ±20%. These transformations imitate natural variation in

patient positioning and scanner settings and prevent the network from memorising individual pictures. Validation and test images are not augmented.

5.6 Model Development Strategy

Three models were developed in sequence, each motivated by the shortcomings of the previous one.

5.6.1 Stage 1: VGG16 Baseline

The baseline uses two-stage transfer learning. In stage 1 the VGG16 convolutional base, initialised with ImageNet weights and frozen, is run once over every image and the resulting feature maps are cached. A small classifier head (Flatten, Dense 256 with ReLU, Dropout 0.5, Dense 4 with softmax) is trained on the cached features for up to 30 epochs with early stopping, which takes only minutes even on a CPU. In stage 2 the last convolutional block (`block5`) is unfrozen and the whole network is fine-tuned for six epochs at a very low learning rate of 1e-5 with live augmentation. The baseline reaches 97.10% accuracy on the test set.

5.6.2 Stage 2: EfficientNet-B0 First Run

To obtain a smaller and faster model, EfficientNet-B0 with ImageNet weights was fully fine-tuned in PyTorch using focal loss ($\gamma = 2$) [7] with class weights and label smoothing 0.1, AdamW [9] with learning rates of 3e-5 for the backbone and 3e-4 for the head, a one-cycle schedule [10], up to 20 epochs and early stopping with patience 5. Evaluated on the test set it scored only 81.7% (82.9% with test-time augmentation), with glioma recall of 73% and meningioma recall of 61%. A second checkpoint (`brain_tumor_efficientnet_best`) supplied by a team-mate scored 80.8% and detected only 18% of gliomas.

5.6.3 Stage 3: Diagnosis and Fine-tuning of the Checkpoint

Before changing anything, the checkpoint was diagnosed. The weakness appeared equally on images from the training folder (a held-out slice of it gave 80.7% accuracy and 17.5% glioma recall), so it was a training problem, not a serving or data-path problem; six alternative pre-processing variants (no crop, no normalisation, other resolutions) gave no improvement. The likely causes were a very low learning rate for the backbone and the combination of focal loss with label smoothing, which together suppress the gradient on hard, ambiguous examples such as gliomas. The remedy was to continue training from the checkpoint with the settings in Table 5.3.

Table 5.3: Fine-tuning Configuration of the Final Model

Setting	Value
Starting point	`brain_tumor_efficientnet_best` checkpoint (EfficientNet-B0, 4-class head)
Training / validation / test images	4,856 / 856 / 1,311 (validation split from Training, seed 42)
Class weights (glioma, meningioma, no tumour, pituitary)	1.083, 1.063, 0.897, 0.979
Loss function	Weighted cross-entropy, label smoothing 0.05 (focal loss removed)
Optimiser	AdamW, weight decay 0.01, gradient clipping at 1.0
Learning rates	1e-4 for the backbone, 5e-4 for the classifier head
Schedule	One-cycle, 15% warm-up, maximum 25 epochs
Early stopping	Patience 6 epochs on validation accuracy + worst-class recall
Batch size / input size	32 / 224×224
Result	Stopped after 19 epochs (best: epoch 13), 89.5 minutes on CPU

A deliberate choice was to select checkpoints by the **sum of validation accuracy and the recall of the worst class**, not by accuracy alone. A model that is excellent overall but fails on one tumour type can therefore never win. Because the starting weights were kept, the model needed only 13 epochs to reach its best validation accuracy instead of the 20 or more normally needed from ImageNet weights.

5.7 Frontend Development Strategy

The interface was developed component by component, starting from the upload panel and the result panel against a stub response, and then connected to the live API. Vite provided fast reloads and the proxy to Flask. Each component receives its data through props, which keeps it easy to test by hand with different results. The project is configured with a linter (oxlint), and the production build is verified with `npm run build`.

5.8 Backend Development Strategy

The back end was built around a stable API contract that was fixed early. The first version served only the VGG16 model; the EfficientNet back-end was added behind the same contract rather than replacing it, which allowed a direct comparison of the models through the same tests. Later changes — a robust checkpoint loader, a thread-safe inference lock, JSON error handlers and safer defaults for debug mode and host binding — were each driven by a concrete observation made during testing.

5.9 Tools Used

Table 5.4: Technology Stack Overview

Layer	Technology	Purpose
Deep learning	PyTorch 2.14, torchvision 0.29	EfficientNet-B0 model, training and Grad-CAM
Baseline model	TensorFlow / Keras, VGG16	Two-stage transfer-learning baseline
Image processing	Pillow, OpenCV, NumPy	Decoding, cropping, colour-mapping and overlays
API	Flask 3.1, Flask-CORS	REST endpoints and error handling
Front end	React 19, Vite 8, Framer Motion, three.js	Single-page interface and animated hero
Evaluation	scikit-learn, Matplotlib	Metrics, confusion matrices and plots
Testing	pytest, requests, Playwright	Unit, API and browser-level checks
Diagrams	Graphviz, Matplotlib	Architecture, UML and flow diagrams
Version control	Git and GitHub	Source and model-artifact management

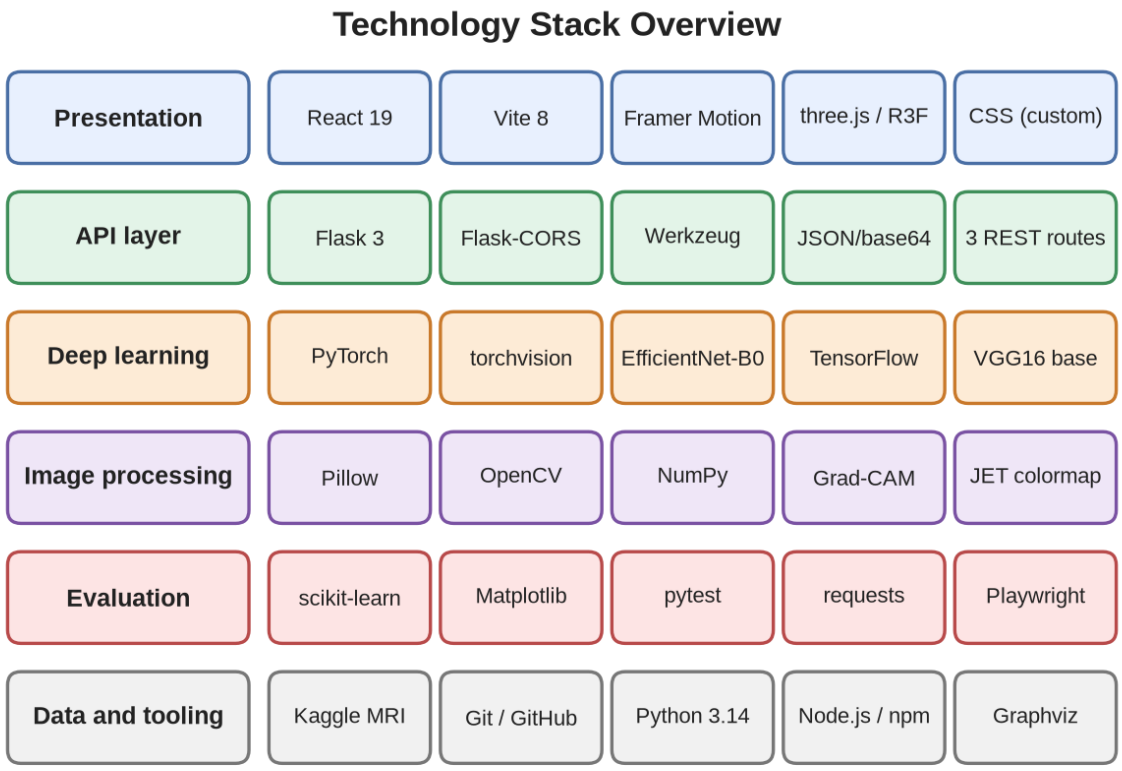


Fig 5.4: Technology Stack Overview

5.10 Testing Strategy

Testing is organised in four layers, described in detail in Chapter 7: **unit tests** for the pre-processing helpers and the checkpoint loader; **API tests** that exercise every route with valid

and invalid input; **system tests** that run the complete 1,311-image test set through the live HTTP API and compare the result with the offline evaluation; and **non-functional tests** for latency and for concurrent users. A browser-level run with Playwright drives the actual interface to confirm that uploads, the heatmap toggle, the history and the error message behave as designed.

5.11 Risk Management

Table 5.5: Risk Register and Mitigation Strategies

Risk	Likelihood	Impact	Mitigation
A class collapses while overall accuracy looks acceptable	Occurred	High	Report per-class recall; select checkpoints on worst-class recall
Train/serve pre-processing mismatch silently lowers accuracy	Medium	High	Single shared crop/resize/normalise definition; full-test-set API evaluation
Concurrent requests corrupt shared Grad-CAM state	Occurred	High	Inference lock; concurrency test with 4 simultaneous users
Very long CPU training time	High	Medium	Cache pre-processed images; fine-tune from a checkpoint; early stopping
Over-fitting to a small dataset	Medium	High	Augmentation, dropout, weight decay, validation-based stopping, untouched test set
Non-MRI or corrupt input gives a misleading answer	Medium	Medium	Type, size and decode checks; documented limitation for out-of-distribution images
Insecure defaults exposed on a network	Medium	High	Debug off by default; localhost binding; 16 MB upload limit
Library version drift between machines	Medium	Low	Pinned requirements; tests that load the checkpoint and run the API

5.12 Gantt Chart

Fig 5.5 shows the planned timeline of the project. Literature study and requirements came first; model experiments started in week 4 and ran in parallel with the back-end and front-end work; fine-tuning, testing and documentation closed the project. The chart shows the plan with durations in weeks; the week numbers are relative to the project start.

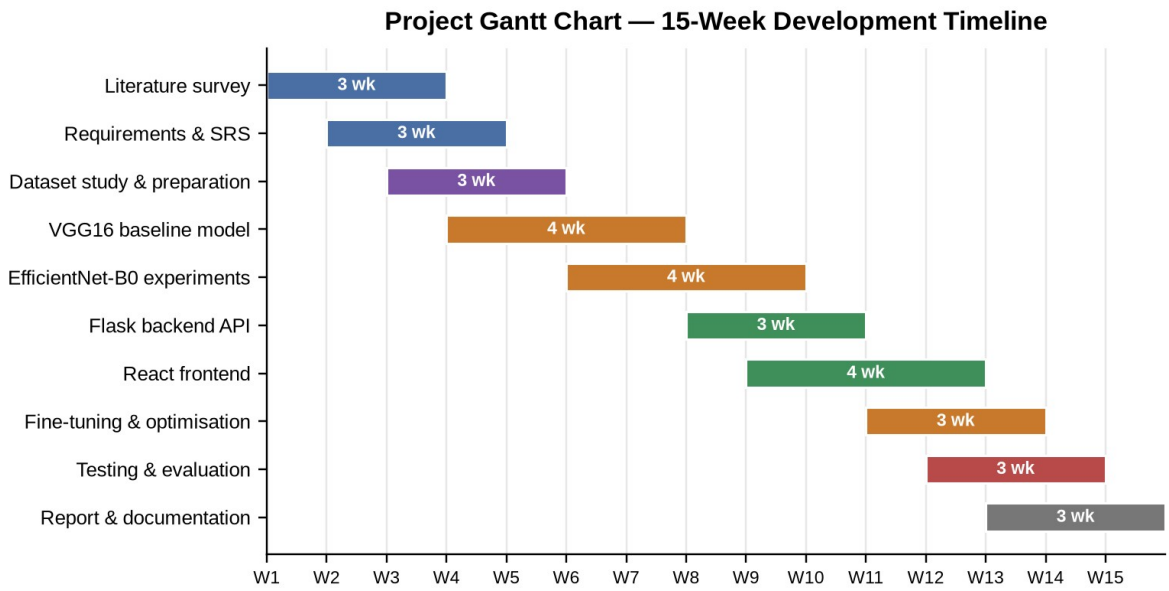


Fig 5.5: Project Gantt Chart — 15-Week Development Timeline

Chapter 6
Implementation

6.1 Introduction

This chapter explains how the design of Chapter 4 was implemented. It covers the environment, the main modules, the model, training and Grad-CAM code, the API, the user interface, the flowcharts of the processing logic, screenshots of the running application and the key code listings. All listings are taken from the project repository and have been shortened only by removing comments and blank lines.

6.2 System Setup and Environment

The project lives in one repository with four main directories: `backend/` (Flask API and PyTorch inference), `ml/` (VGG16 training and TensorFlow Grad-CAM), `new model/` (EfficientNet training, fine-tuning and evaluation scripts) and `frontend/` (React application). A Python virtual environment holds PyTorch, torchvision, Flask, Flask-CORS, Pillow, OpenCV, NumPy, scikit-learn, Matplotlib and pytest; the front end uses npm for React, Vite, Framer Motion and three.js. The back end starts with `python backend/app.py` and listens on port 5000; the front end starts with `npm run dev` and serves the interface on port 5173, proxying `/api` calls to Flask. Table 6.1 summarises the modules.

Table 6.1: Module-wise Description of the System

Module	File(s)	Responsibility
API server	backend/app.py	Routes, validation, model selection, inference lock, JSON responses
Inference + Grad-CAM	backend/efficientnet_infer.py	Crop, normalise, forward pass, Grad-CAM and heatmap overlay (PyTorch)
VGG16 baseline	ml/train.py, ml/gradcam.py	Two-stage training; TensorFlow Grad-CAM
EfficientNet training	new model/main.py	First EfficientNet-B0 training run (focal loss)
Fine-tuning	new model/finetune_best.py	Fine-tunes the checkpoint; writes the v2 model, report and plots
Offline evaluation	new model/evaluate_best.py	Evaluates a checkpoint on the test set; confusion matrix
API evaluation	ml/evaluate_api.py	Sends every test image to the live API and reports accuracy
Automated tests	backend/tests/test_backend.py	18 unit and API tests
User interface	frontend/src/*	Upload, result display, history, hero animation

6.3 Dataset Loading and Pre-processing Implementation

Pre-processing is implemented once in `backend/efficientnet_infer.py` and mirrored in the training scripts. Listing 6.1 shows the crop function: it thresholds the grey-scale image at 10, finds the bounding box of the non-black pixels and pads it by four pixels. The function returns a box rather than a cropped image so that the Grad-CAM map computed on the crop can be pasted back at the correct position in the full image. During fine-tuning the crop and resize are applied once to all 5,712 training images and the results are cached in a NumPy array of 224×224×3 bytes per image (about 860 MB), which removes the cost of decoding large JPEG files on every epoch.

Listing 6.1: Brain-region crop (backend/efficientnet_infer.py)

```
def crop_brain_region_bbox(img, threshold=10, padding=4):
    """Bounding box (x0, y0, x1, y1) of the non-black brain tissue region."""
    gray = np.array(img.convert("L"))
    mask = gray > threshold
    if mask.sum() == 0:
        return 0, 0, img.width, img.height
    coords = np.argwhere(mask)
    y0, x0 = coords.min(axis=0)
    y1, x1 = coords.max(axis=0) + 1
    h, w = gray.shape
    y0, x0 = max(0, y0 - padding), max(0, x0 - padding)
    y1, x1 = min(h, y1 + padding), min(w, x1 + padding)
    return int(x0), int(y0), int(x1), int(y1)
```

6.4 Model Implementation

The EfficientNet-B0 architecture is taken from torchvision with its classifier replaced by dropout (p = 0.4) and a linear layer with four outputs. Listing 6.2 shows the builder and the checkpoint loader. The loader accepts either a normal `.pth` file or an *extracted* checkpoint folder: when a `.pth` archive has been unzipped into `data.pkl` and a `data/` directory, `torch.load` cannot read it directly, so the loader re-packs the files into an in-memory zip archive and loads that. This allowed the team-mate’s checkpoint folder to be used without any manual conversion.

Listing 6.2: Model builder and checkpoint loader (backend/efficientnet_infer.py)

```
def build_model(num_classes):
    model = models.efficientnet_b0(weights=None)
    num_features = model.classifier[1].in_features          # 1280
    model.classifier[1] = nn.Sequential(
        nn.Dropout(p=0.4, inplace=True),
        nn.Linear(num_features, num_classes),
    )
    return model


def _load_state_dict(checkpoint_path):
    """Load a .pth file, or an extracted .pth folder (data.pkl, data/0...)."""
    if not os.path.isdir(checkpoint_path):
        return torch.load(checkpoint_path, map_location="cpu")
    root = os.path.basename(os.path.normpath(checkpoint_path))
    buf = io.BytesIO()
    with zipfile.ZipFile(buf, "w", zipfile.ZIP_STORED) as z:
        for dirpath, _, filenames in os.walk(checkpoint_path):
            for fname in filenames:
                full = os.path.join(dirpath, fname)
                arc = f"{root}/{os.path.relpath(full, checkpoint_path)}"
                with open(full, "rb") as f:
                    z.writestr(arc, f.read())
    buf.seek(0)
    return torch.load(buf, map_location="cpu")
```

The VGG16 baseline is implemented in TensorFlow/Keras in `ml/train.py`. Stage 1 caches the bottleneck features of the frozen base and trains the dense head; stage 2 unfreezes `block5` and fine-tunes at a learning rate of 1e-5 with live augmentation. The model is saved as two files, `vgg16_base.keras` and `classifier_head.keras`, for the reason explained in Chapter 4.

6.5 Fine-tuning Implementation

The fine-tuning script `new model/finetune_best.py` follows the workflow in Fig 6.1. It loads the starting checkpoint, caches the images, splits off the validation set, and then trains with the settings of Table 5.3. The core of the loop is shown in Listing 6.3: weighted cross-entropy with label smoothing, AdamW with separate learning rates for backbone and head, a one-cycle scheduler, gradient clipping, and a checkpoint rule based on the validation accuracy plus the recall of the worst class. When validation stops improving for six epochs the loop ends, the best weights are restored and the untouched test set is evaluated once, producing the report, confusion matrix and training curves.

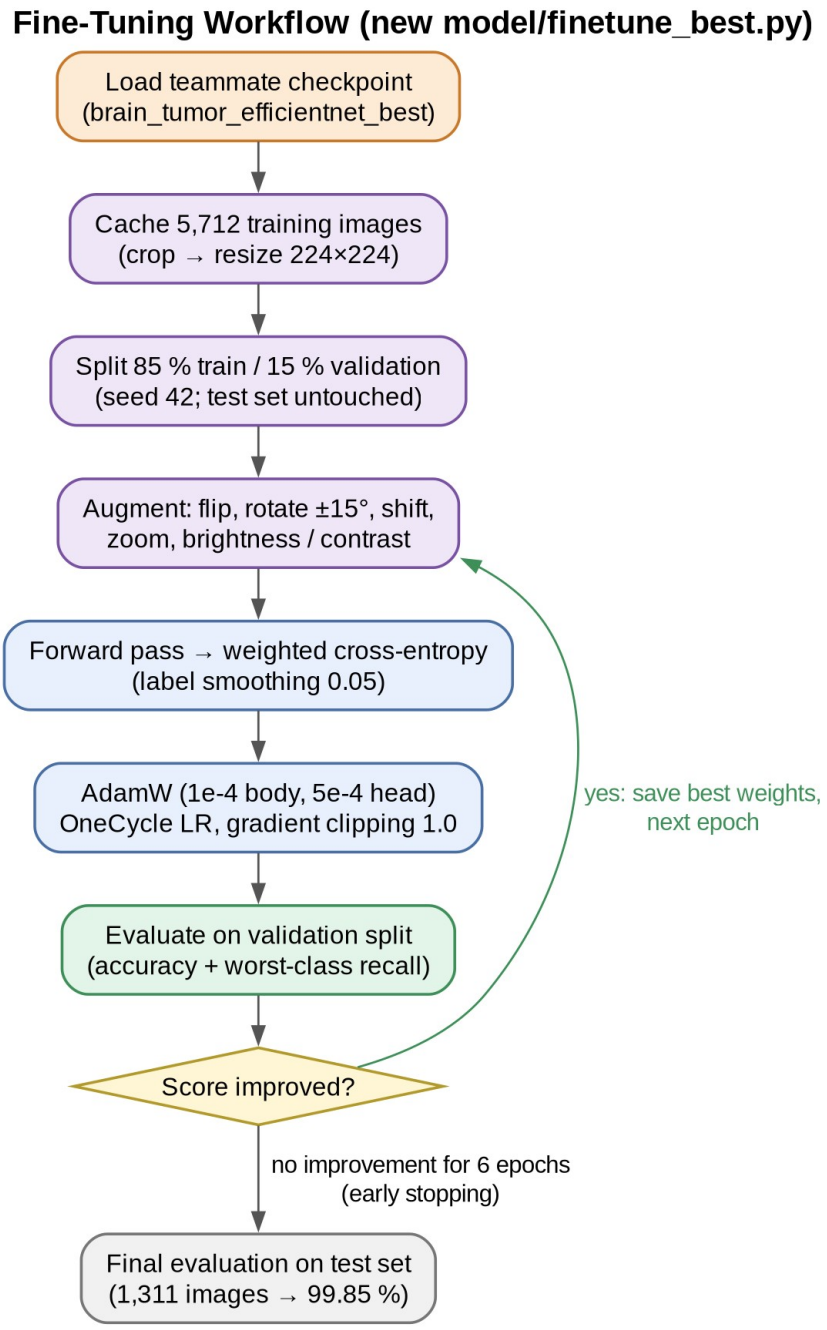


Fig 6.1: Execution Flow of the Fine-Tuning Script

Listing 6.3: Core of the fine-tuning loop (new model/finetune_best.py)

```
criterion = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.05)
optimizer = optim.AdamW([{"params": backbone, "lr": 1e-4},
                         {"params": head, "lr": 5e-4}], weight_decay=0.01)
scheduler = optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=[1e-4, 5e-4], epochs=epochs,
    steps_per_epoch=len(train_loader), pct_start=0.15)
for epoch in range(epochs):
    model.train()
    for x, y in train_loader:
        optimizer.zero_grad()
        loss = criterion(model(x), y)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        scheduler.step()

    p, l = predict(model, val_loader)
    val_acc = float((p == l).mean())
    rec = per_class_recall(l, p, len(class_names))
    score = val_acc + min(rec)            # a collapsed class can never win
    if score > best_score:
        best_score, no_improve = score, 0
        torch.save(model.state_dict(), CHECKPOINT_PATH)
    else:
        no_improve += 1
        if no_improve >= patience:
            break
```

6.6 Grad-CAM Implementation

The `GradCAM` class in Listing 6.4 registers a forward hook on the target layer to store its activations and a full backward hook to store the gradients that flow into it. `generate()` runs a forward pass, back-propagates the score of the predicted class, averages the gradients over each 7×7 channel to obtain a weight, forms the weighted sum of the 1,280 activation maps, applies ReLU and rescales the result to [0, 1]. The surrounding function `predict_with_gradcam()` then resizes the map to the crop box in display coordinates, and `overlay_heatmap_on_image()` blends it over the image with the JET colour map at 40% opacity.

Listing 6.4: Grad-CAM for EfficientNet-B0 (backend/efficientnet_infer.py)

```
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.activations = self.gradients = None
        self.handles = [
            target_layer.register_forward_hook(self._save_activation),
            target_layer.register_full_backward_hook(self._save_gradient),
        ]

    def generate(self, input_tensor, class_idx=None):
        self.model.zero_grad()
        output = self.model(input_tensor)
        if class_idx is None:
            class_idx = int(torch.argmax(output[0]))
        output[0, class_idx].backward()

        weights = self.gradients[0].mean(dim=(1, 2))   # per-channel weight
        cam = (weights[:, None, None] * self.activations[0]).sum(dim=0)
        cam = F.relu(cam)
        cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
        probs = torch.softmax(output, dim=1)[0].detach().cpu().numpy()
        return cam.cpu().numpy(), class_idx, probs
```

6.7 API Development

The `/api/predict` route in Listing 6.5 performs the validation steps of Fig 6.2 before touching the model: a missing field, an unsupported extension and an unreadable image each return HTTP 400 with a JSON message, and a request above 16 MB is rejected with 413 by a dedicated error handler. The model call is wrapped in an inference lock, discussed in Section 6.8. The response contains the predicted class, a boolean `tumor_detected`, the confidence, the four probabilities and the two images as base64 PNG data URIs, so the interface needs no second request to fetch the heatmap.

Listing 6.5: The /api/predict endpoint (backend/app.py)

```
@app.route("/api/predict", methods=["POST"])
def predict():
    _load_models()
    if _state["load_error"]:
        return jsonify({"error": _state["load_error"]}), 503
    if "image" not in request.files:
        return jsonify({"error": "No image file provided."}), 400
    file = request.files["image"]
    if file.filename == "" or not allowed_file(file.filename):
        return jsonify({"error": "Please upload a PNG, JPG or BMP image."}), 400
    try:
        raw = Image.open(file.stream).convert("RGB")
    except Exception:
        return jsonify({"error": "Could not read the uploaded file."}), 400

    display_img = resize_for_display(raw)
    with _infer_lock:              # Grad-CAM keeps state on shared hooks
        heatmap, pred_idx, probs = predict_with_gradcam(
            raw, display_img, _state["gradcam"], class_names)
    pred_class = class_names[pred_idx]
    return jsonify({
        "prediction": pred_class,
        "tumor_detected": pred_class != "notumor",
        "confidence": float(probs[pred_idx]),
        "probabilities": {c: float(p) for c, p in zip(class_names, probs)},
        "original_image": image_to_data_uri(display_arr),
        "heatmap_overlay": image_to_data_uri(overlay_arr),
    })
```

6.8 Thread-Safety of Inference

Flask’s development server handles each request on its own thread, but the Grad-CAM object keeps its activations and gradients on the instance, and PyTorch’s autograd state is shared by the loaded model. When a benchmark first sent four simultaneous requests to the service, the process crashed with a memory-corruption error (`free(): chunks in smallbin corrupted`). The fix, shown in Listing 6.6, serialises model loading and every inference with a re-entrant lock. After the change the same benchmark completed 100 concurrent requests with no errors, and Chapter 7 reports the resulting latencies.

Listing 6.6: Serialising inference with a lock (backend/app.py)

```
_infer_lock = threading.RLock()


def _load_models():
    with _infer_lock:          # loading is idempotent and thread-safe
        _load_models_locked()


# every inference call is wrapped the same way:
with _infer_lock:
    heatmap, pred_idx, probs = predict_with_gradcam(
        raw, display_img, gradcam, class_names)
```

6.9 UI Development using React

The user interface is composed of the components described in Chapter 4. `UploadPanel` validates the file type on the client (PNG, JPEG or BMP), creates an object URL for the preview and calls `onAnalyze(file)`. `App` implements the handler shown in Listing 6.7: it sets the loading flag, calls `predictImage`, prepends the result to the history and makes it active, or stores the error message returned by the API. `ResultPanel` chooses the banner colour from `tumor_detected`, shows the confidence and a one-line description of the predicted class, and swaps the image between `original_image` and `heatmap_overlay` when the toggle is pressed. `ProbabilityBars` draws one bar per class with its percentage, and `HistoryPanel` lists previous results as thumbnails.

Listing 6.7: Calling the API and updating the interface (frontend/src)

```
// api.js
export async function predictImage(file) {
  const formData = new FormData();
  formData.append("image", file);
  const res = await fetch(`${API_BASE}/predict`,
                          { method: "POST", body: formData });
  return parseJsonOrThrow(res);     // throws Error(body.error) on 4xx / 5xx
}

// App.jsx
async function handleAnalyze(file) {
  setLoading(true);
  setError(null);
  try {
    const res = await predictImage(file);
    const entry = { id: nextId++, ...res };
    setHistory((prev) => [entry, ...prev]);  // newest first
    setActiveId(entry.id);
  } catch (err) {
    setError(err.message || "Something went wrong.");
  } finally {
    setLoading(false);
  }
}
```

6.10 Flowcharts for System Logic

Fig 6.2 summarises the decision logic of one analysis. Invalid input leaves through one of the two error branches with an HTTP 400; a valid image is cropped, resized, normalised and classified, and the predicted class decides whether the green “no tumour” or the red “possible tumour” banner is shown. In both cases the Grad-CAM overlay and the probability bars are displayed.

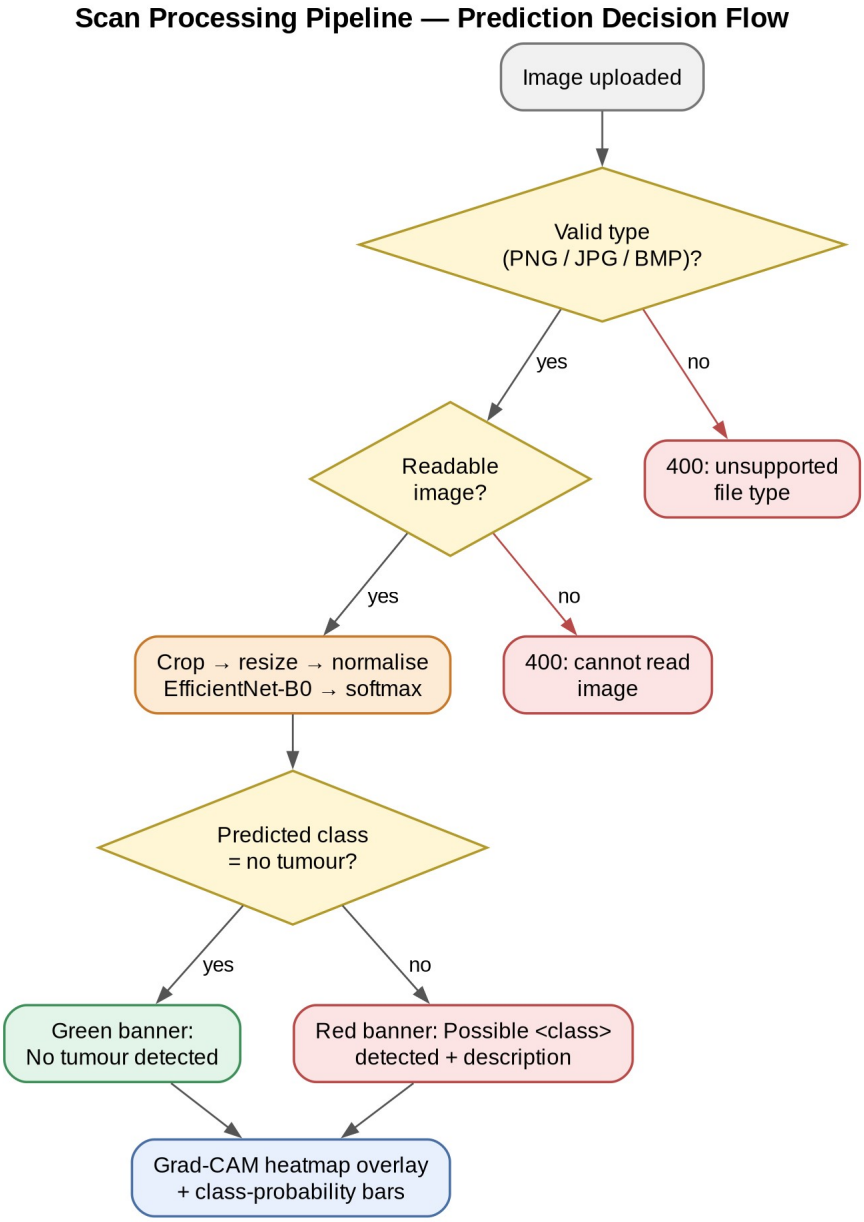


Fig 6.2: Scan Processing Pipeline — Prediction Decision Flow

6.11 Screenshots of Application

The following screenshots were captured from the running application. They were produced by driving the interface in a browser with the test images of the dataset, using the final fine-tuned model. Fig 6.3 shows the landing page, whose headline figure (99.8%) is read from the model's evaluation report through `/api/model-info`.

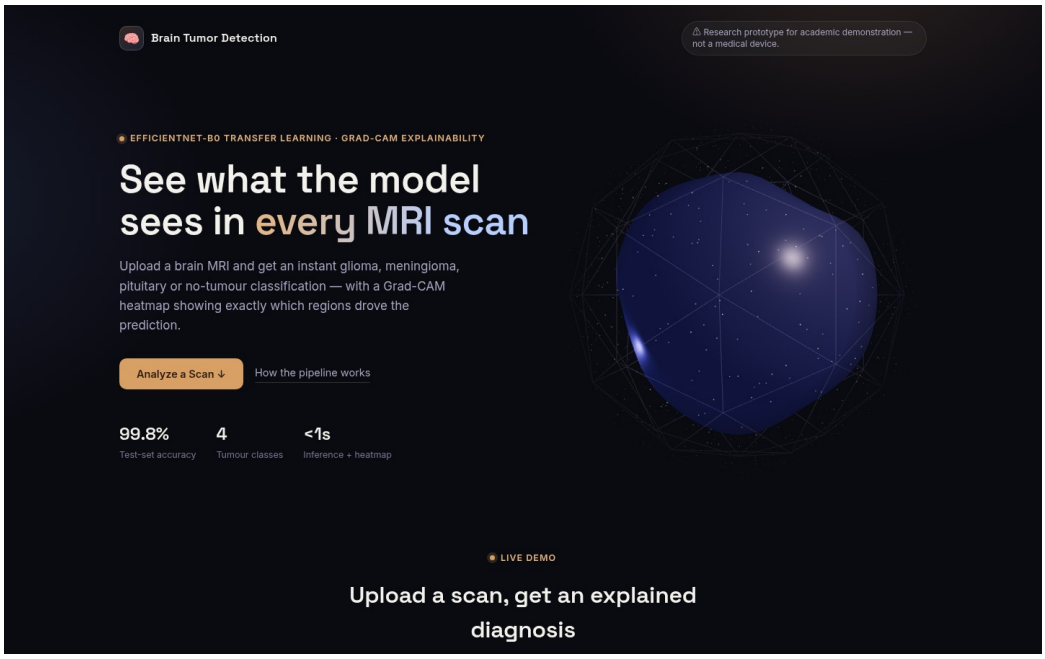


Fig 6.3: Home Page — Hero Section with Animated 3D Brain and Test-set Accuracy

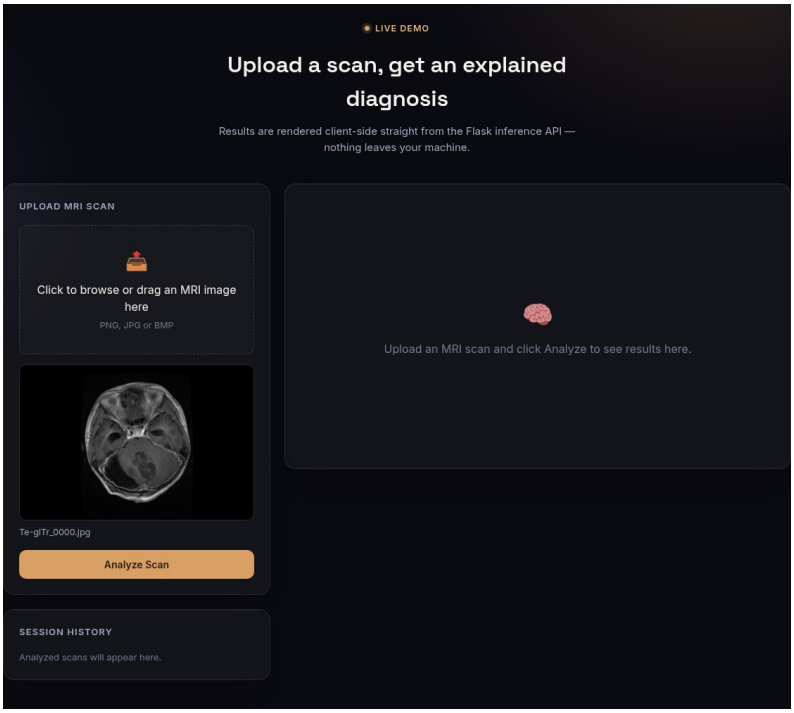


Fig 6.4: Upload Panel — Scan Selected and Previewed Before Analysis

After **Analyze Scan** is pressed, the result panel in Fig 6.5 appears. For a glioma slice the banner is red, the confidence is 96.8%, and the Grad-CAM view concentrates on the tumour in the posterior fossa. The toggle in Fig 6.6 switches back to the unmodified scan, which lets the user compare the highlighted area with the underlying anatomy.

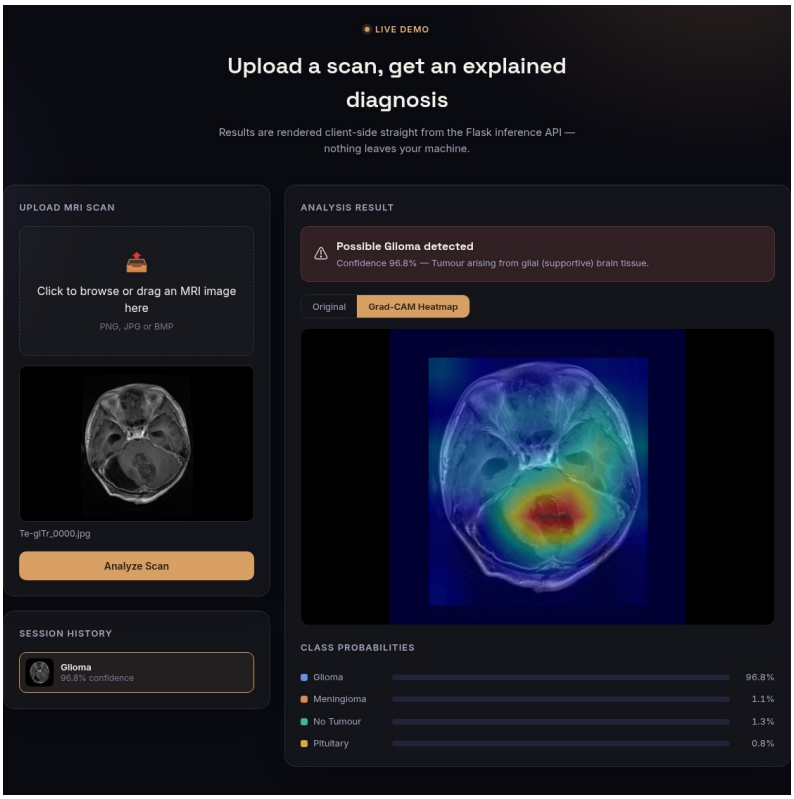


Fig 6.5: Glioma Result with Grad-CAM Heatmap and Class Probabilities

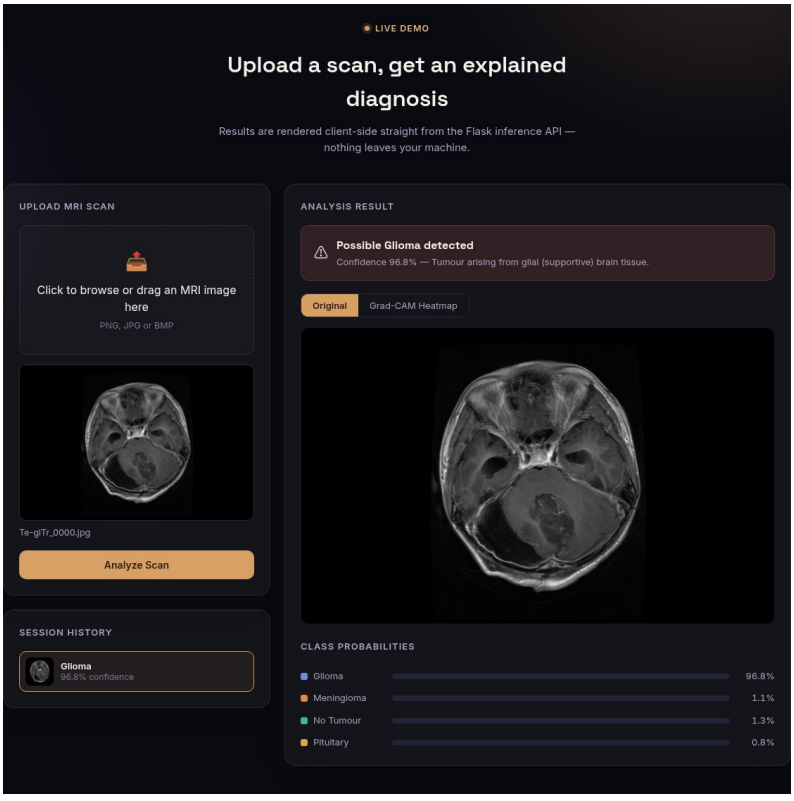


Fig 6.6: Same Result in the Original Image View

Fig 6.7 and Fig 6.8 show the other two tumour classes. In both cases the attention map covers the visible lesion, and the probability bars show a clearly dominant class. Fig 6.9 shows a

healthy scan: the banner turns green and the heatmap is spread symmetrically over the central brain structures instead of forming a focus.

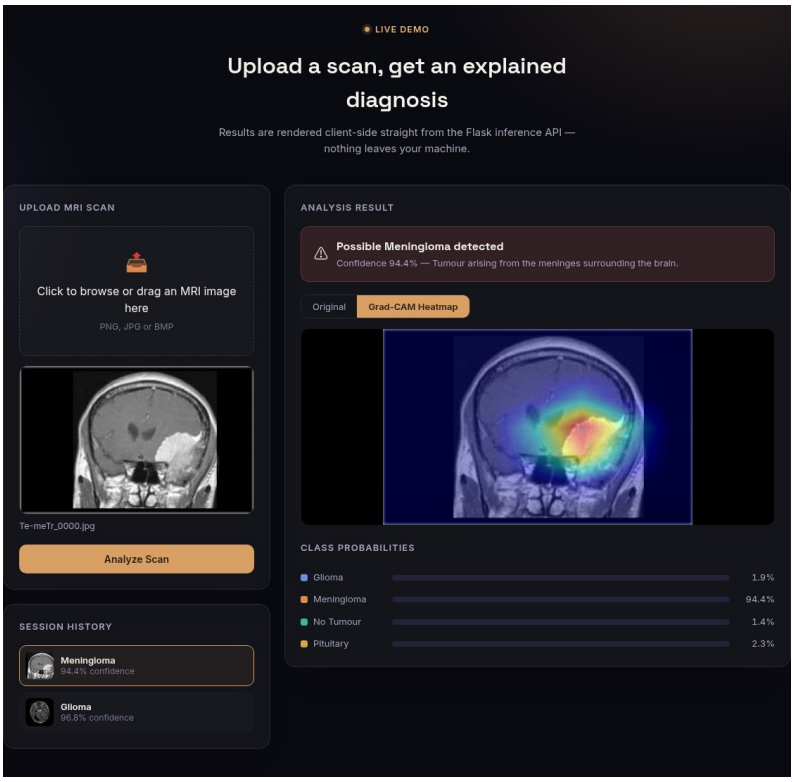


Fig 6.7: Meningioma Result (confidence 94.4%)

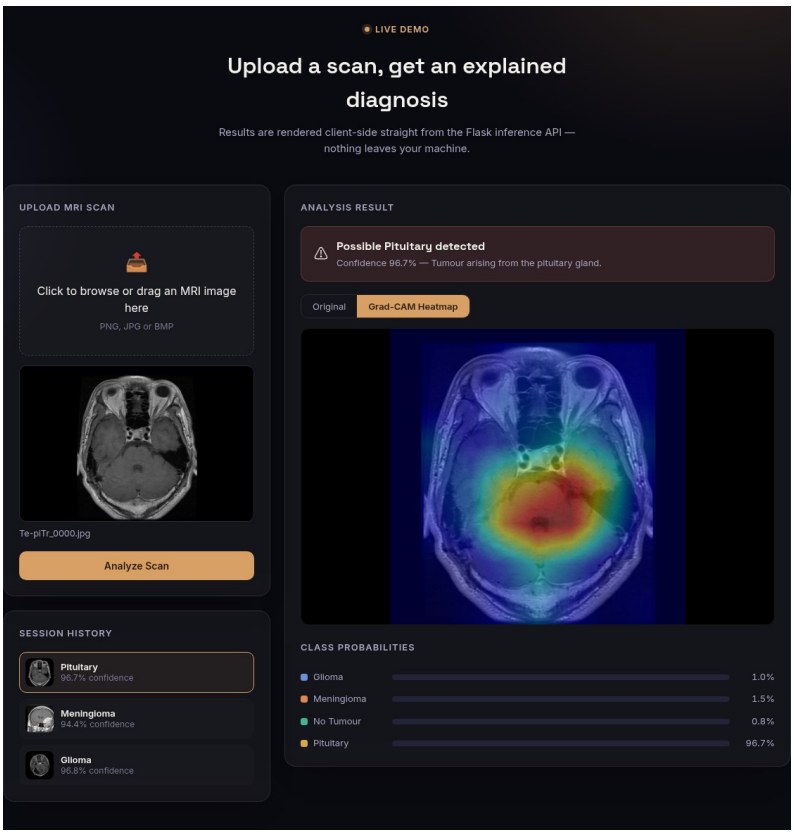


Fig 6.8: Pituitary Tumour Result (confidence 96.7%)

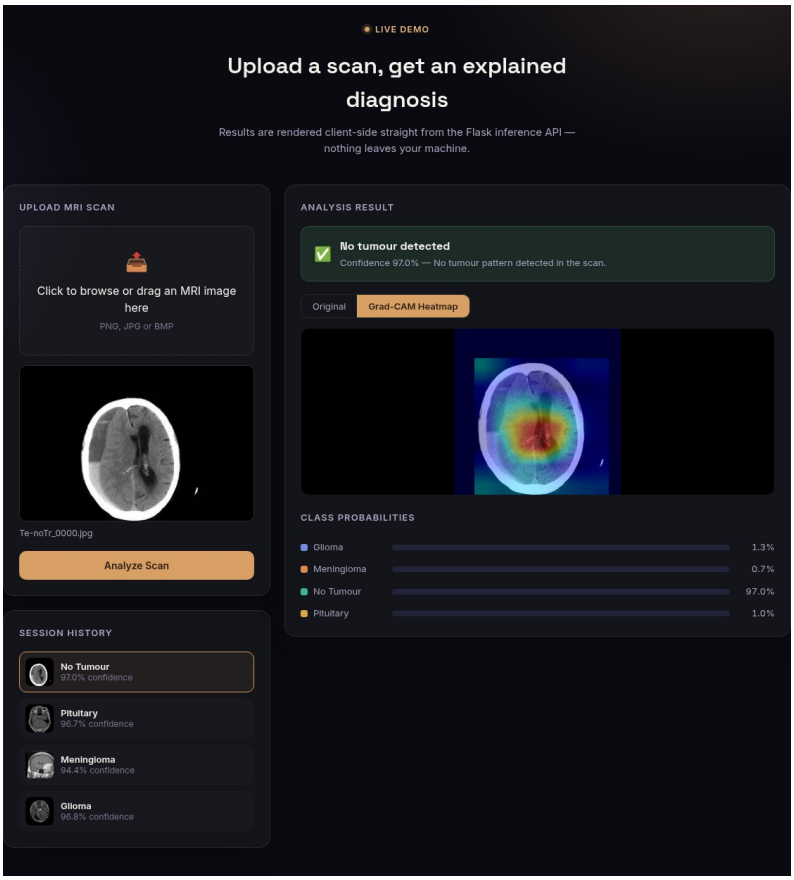


Fig 6.9: No-Tumour Result (confidence 97.0%)

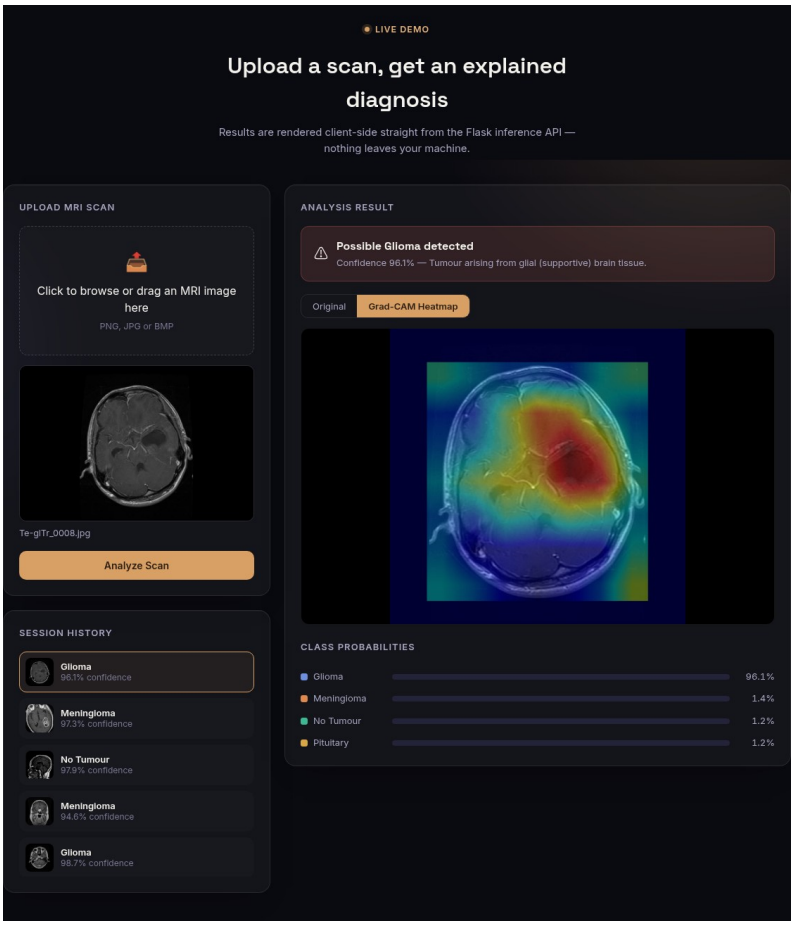


Fig 6.10: Session History after Analysing Five Scans

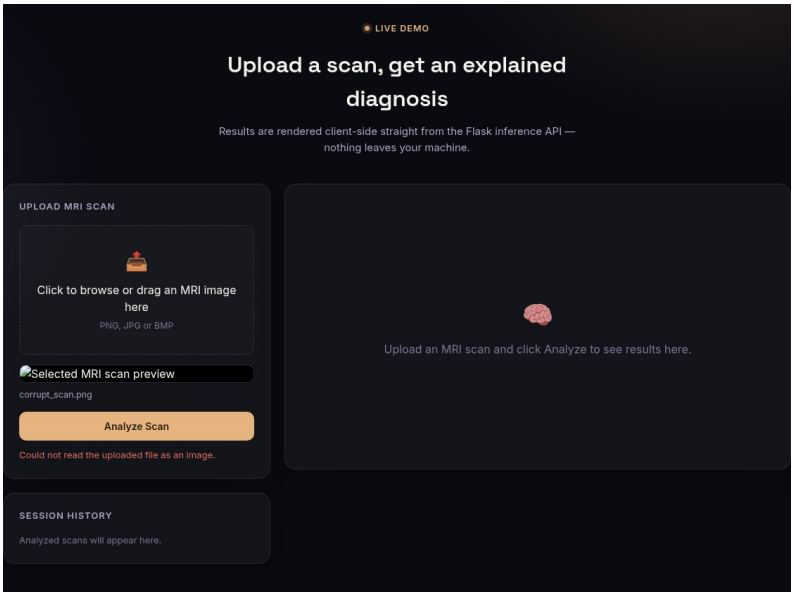


Fig 6.11: Error Message Shown When the Server Cannot Read the Uploaded File

6.12 Key Code Snippets

The listings in this chapter are the parts of the code that carry the project's main ideas: the shared pre-processing (Listing 6.1), the model and checkpoint loader (Listing 6.2), the

checkpoint selection rule in fine-tuning (Listing 6.3), Grad-CAM (Listing 6.4), the prediction endpoint (Listing 6.5), the inference lock (Listing 6.6) and the client-side call (Listing 6.7). Taken together they are fewer than 250 lines, which reflects how much of the system's behaviour comes from pre-trained components and carefully chosen training settings rather than from custom code.

Chapter 7
Testing

7.1 Introduction

This chapter reports how the system was verified. The tests cover four questions: does each component behave correctly (unit tests), does the whole service behave correctly for valid and invalid requests (API and functional tests), is the model accurate on data it has never seen (system and results tests), and is the service fast and safe under load (performance and concurrency tests). Every number in this chapter was produced by a script or test that is part of the repository, so it can be reproduced.

7.2 Testing Strategy

The strategy follows the layers of the architecture. Pre-processing helpers and the checkpoint loader are tested in isolation. The Flask routes are tested through Flask's test client with real images from the dataset and with deliberately broken input. The deployed behaviour is tested over real HTTP against a running server: first with the complete test set, then with a load generator. Finally, the user interface is exercised through a headless Chromium browser driven by Playwright, which uploads images, presses the buttons and captures the screenshots shown in Chapter 6. A principle applied throughout is that the **test set is evaluated only for reporting**: no model, threshold or checkpoint was selected on it.

7.3 Unit Testing

The automated suite in `backend/tests/test_backend.py` contains 18 tests, summarised in Table 7.1. All pass; Fig 7.7 shows the output of the run. The tests check the crop function on a bright rectangle, on an all-black image and with padding that would exceed the image border; the overlay function's output shape and type; the extension filter; the model's four-class head; and that the extracted checkpoint folder can be loaded.

Table 7.1: Automated Test Suite (pytest)

Test group	Tests	What is verified	Result
Pre-processing helpers	5	Crop bounding box, all-black image, padding clamp, overlay shape/dtype, file-extension filter	5 passed
Model loading	2	Four-class head; loading of the extracted checkpoint folder	2 passed
Information routes	2	/api/health reports model loaded; /api/model-info lists classes and metrics	2 passed

Test group	Tests	What is verified	Result
Prediction per class	4	One real test image per class is classified correctly; probabilities sum to 1; heatmap PNG returned	4 passed
Input validation	5	Missing file, wrong extension, corrupt image, tiny black image, 17 MB upload (JSON 413)	5 passed
Total	**18**		**18 passed**

7.4 Integration Testing

Integration testing asks whether the model that was evaluated offline is the same model that the website uses. The script `ml/evaluate_api.py` sends each of the 1,311 test images as a separate HTTP request to the running API, exactly as the browser would, and compares the answers with the labels. For the final model it produced the confusion matrix shown in Fig 7.8: **99.85% accuracy, identical to the offline evaluation**, with no failed requests. The same script applied to the VGG16 back-end earlier gave 97.10%, also identical to its offline figure. Agreement of the two numbers shows that pre-processing, class ordering and image encoding in the serving path introduce no loss of accuracy.

7.5 System Testing

System testing evaluates the final model on the 1,311-image test set. Table 7.2 gives precision, recall and F1 for each class, Fig 7.1 the confusion matrix and Fig 7.2 the same metrics as a chart. Of 1,311 images, 1,309 are classified correctly. The two errors are one glioma and one pituitary tumour that were labelled meningioma. No healthy scan was labelled as a tumour and no tumour was labelled healthy, which matters most for a screening tool. The 95% Wilson confidence interval for the accuracy is 99.45%–99.96%, which does not overlap with that of the VGG16 baseline (96.05%–97.88%), so the improvement is not a matter of chance.

Table 7.2: Per-class Results of the Final Model on the Test Set

Class	Precision	Recall	F1-score	Support
Glioma	1.0000	0.9967	0.9983	300
Meningioma	0.9935	1.0000	0.9967	306
No tumour	1.0000	1.0000	1.0000	405
Pituitary	1.0000	0.9967	0.9983	300
Macro average	**0.9984**	**0.9983**	**0.9984**	**1,311**
Accuracy			**0.9985**	**1,311**

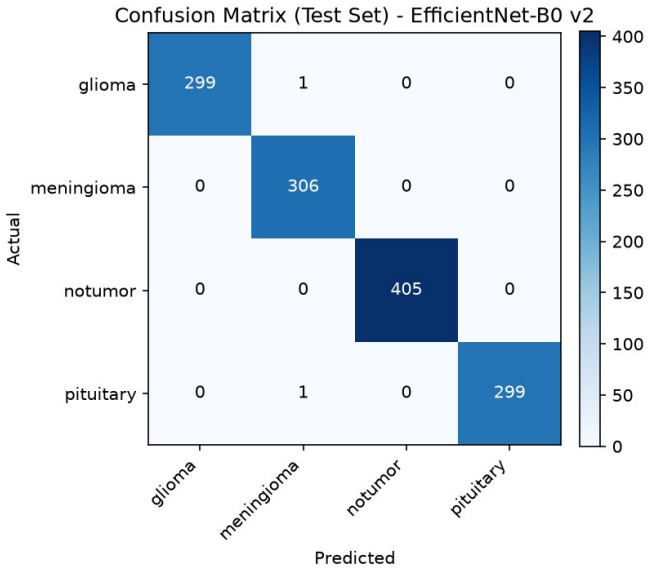


Fig 7.1: Confusion Matrix of the Fine-tuned EfficientNet-B0 on the Test Set

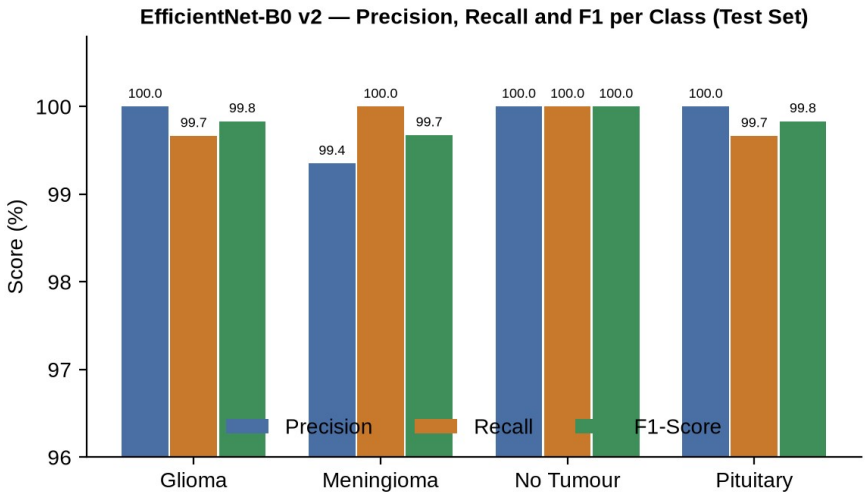


Fig 7.2: Precision, Recall and F1-score per Class (Test Set)

7.6 Functional Test Cases

Table 7.3 lists the functional tests of the user interface. They were run in a browser session against the final model; the file names are images from the dataset's test folder, and the confidence values are those displayed by the interface.

Table 7.3: Functional Test Cases

Test ID	Action / input	Expected and observed result	Status
FT-01	Upload glioma slice Te-glTr_0000.jpg and press Analyze Scan	Red banner "Possible Glioma detected", confidence 96.8%	PASS
FT-02	Upload meningioma slice Te-meTr_0000.jpg	"Possible Meningioma detected", confidence 94.4%	PASS

Test ID	Action / input	Expected and observed result	Status
FT-03	Upload pituitary slice Te-piTr_0000.jpg	“Possible Pituitary detected”, confidence 96.7%	PASS
FT-04	Upload healthy slice Te-noTr_0000.jpg	Green banner “No tumour detected”, confidence 97.0%	PASS
FT-05	Press Original and Grad-CAM Heatmap in turn	The image switches between the scan and the overlay	PASS
FT-06	Read the class probability bars	Four bars; values (96.8, 1.1, 1.3, 0.8%) sum to 100%	PASS
FT-07	Analyse five different scans in a row	History lists five thumbnails, newest first; clicking one reopens it	PASS
FT-08	Choose a .txt file in the upload box	File ignored by the client; preview unchanged	PASS
FT-09	Upload a text file named corrupt_scan.png	Red message “Could not read the uploaded file as an image.”	PASS
FT-10	Open the landing page with the backend running	Hero shows test-set accuracy 99.8% read from /api/model-info	PASS

7.7 Invalid Input and Robustness Testing

Table 7.4 lists the requests sent directly to the API to probe its error handling. Every invalid request received a clear JSON error with the correct HTTP status; none crashed the server. One defect was found: an upload above 16 MB originally received Flask’s default HTML 413 page, so the interface could display only a generic message. A JSON error handler was added and a regression test written for it (part of the input-validation group in Table 7.1).

Table 7.4: Invalid-Input and Robustness Test Results

Request	Expected behaviour	Observed response
GET /api/predict	Method not allowed	405
POST without an image field	400 with explanation	400 “No image file provided (expected form field ‘image’).”
notes.txt (text file)	Reject unsupported type	400 “Please upload a PNG, JPG or BMP image.”
Text content named a.png	Reject undecodable image	400 “Could not read the uploaded file as an image.”
17 MB random data	Reject oversized upload	413 “File is too large (maximum 16 MB).”
300×300 greyscale PNG	Accept and classify	200 (converted to RGB)
64×64 RGBA PNG	Accept and classify	200 (alpha channel dropped)
1×1 pixel image	Accept without crashing	200
3000×3000 JPEG	Accept, down-scale for display	200 in 0.29 s

The last four rows also expose a limitation: the system accepts any decodable image and always returns one of the four labels. A blank or non-MRI picture therefore receives a prediction (it is usually classified as “no tumour”). The interface states that the system is for

MRI slices only, but an out-of-distribution detector would be needed to reject other images automatically.

7.8 Concurrency Testing and the Thread-Safety Defect

A load test with four simultaneous users exposed a serious defect. Flask's development server runs each request on its own thread, but the Grad-CAM object stores activations and gradients on the instance and the model is shared. Under concurrent requests the server process crashed with the message `free(): chunks in smallbin corrupted`, which indicates memory corruption. The remedy was a re-entrant lock around model loading and around every inference (Listing 6.6). After the change the same test completed 100 concurrent requests with no failure, as Table 7.5 shows. The test suite now includes the full-API run, and the lock is documented in the code so that future changes keep inference serialised.

7.9 Performance Testing

Latency was measured against the live server with 100 randomly chosen test images per scenario: one user sending requests one after another, and four users sending them at the same time. The times include the HTTP transfer, validation, pre-processing, the forward and backward passes, the heatmap overlay and the encoding of two PNG images (about 361 KB of response). All figures were obtained on a CPU-only laptop (Intel Core i7-12650H).

Table 7.5: Performance Testing Results (/api/predict, 100 requests per scenario)

Scenario	Mean	P50	P95	Max	Errors
1 user, sequential	141 ms	148 ms	197 ms	219 ms	0
4 concurrent users	298 ms	305 ms	409 ms	522 ms	0
/api/health	1.6 ms	—	1.8 ms	—	0
Full test set (1,311 images), EfficientNet-B0	165 ms / image	—	—	—	0
Full test set (1,311 images), VGG16 back-end	2.75 s / image	—	—	—	0

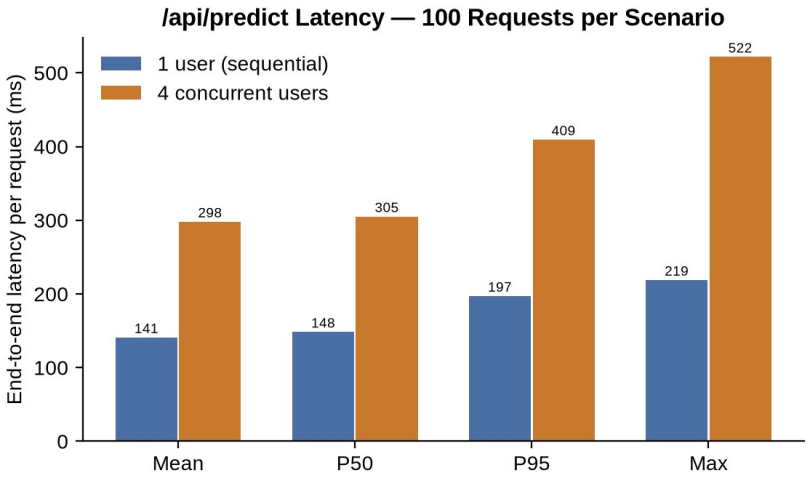


Fig 7.3: /api/predict Latency — One User vs. Four Concurrent Users

A single user receives a complete answer in about 0.15 s, comfortably inside the target of one second, and four simultaneous users still see a 95th-percentile time of 0.41 s, because their requests are queued by the lock. The throughput under four users was 13.2 requests per second. The EfficientNet back-end is about seventeen times faster per image than the VGG16 back-end in the same API evaluation.

7.10 Overfitting and Generalisation Analysis

A model can score well on the data it is trained on and fail on new data; the project therefore checked for over-fitting in three ways. First, Table 7.6 shows that the accuracy on the training images (which are augmented, and therefore harder than plain images) and the accuracy on the validation images rise together and end within a fraction of a percentage point of each other, with no widening gap. Second, validation accuracy stays between 99.5% and 99.9% over the last twelve epochs instead of falling. Third, the test set, which played no part in training or checkpoint selection, gave 99.85%, consistent with the validation result.

Table 7.6: Fine-tuning Progress (selected epochs)

Epoch	Train loss	Train acc.	Val. acc.	Glioma recall (val.)
Start (checkpoint)	—	—	80.7%	17.5%
1	0.4529	90.7%	95.6%	81.0%
2	0.3428	95.0%	98.3%	94.5%
3	0.3057	96.3%	99.0%	97.0%
5	0.2679	97.7%	99.2%	98.0%
7	0.2472	98.8%	99.7%	100%
9	0.2350	99.1%	99.8%	100%
13 (best)	**0.2190**	**99.6%**	**99.9%**	**100%**
19 (stopped)	0.2119	99.8%	99.7%	99.5%

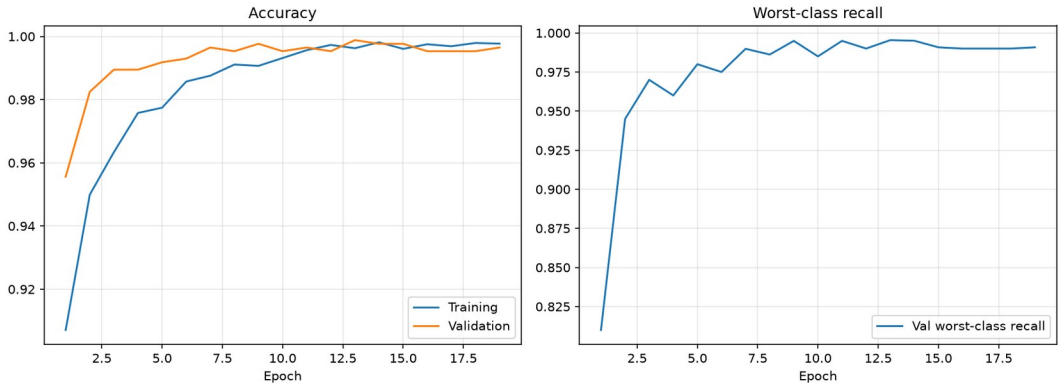


Fig 7.4: Training Curves of the Fine-tuned Model (accuracy and worst-class recall)

Two cautions apply. The Kaggle dataset is compiled from several public collections of multi-slice MRI studies, in which neighbouring slices of the same patient look very similar. Whether a random split keeps all slices of one patient on one side of the split cannot be verified from the dataset itself, and any overlap would make test accuracy higher than it would be on scans from a new hospital. The test result therefore shows that the model has learned this dataset very well, not that it will reach 99.85% in clinical use. The result also comes from a single training run, so the exact figure could vary by a few images with a different random seed.

7.11 Results and Analysis

Table 7.7 compares the four models that were trained or received during the project on the same test set, and Fig 7.5 shows their per-class recall. The fine-tuned EfficientNet-B0 is the best in every class, is the smallest of the accurate models and is the fastest to serve. The comparison also illustrates why overall accuracy is not enough: the team-mate's checkpoint and the first EfficientNet run both score about 81% overall but fail on different classes (gliomas, and gliomas with meningiomas respectively).

Table 7.7: Comparison of Expected vs. Actual Results across Models

Model	Parameters	Test accuracy	Glioma recall	Min. class recall	Time per image (API)
EfficientNet-B0, first run	4.0 M	81.7%	73.0%	61.4%	—
Team-mate checkpoint	4.0 M	80.8%	18.3%	18.3%	—
VGG16 two-stage baseline	21.1 M	97.10%	92.7%	92.7%	2.75 s
EfficientNet-B0 fine-tuned (final)	**4.0 M**	**99.85%**	**99.7%**	**99.7%**	**0.165 s**
Target (NFR-01, NFR-02, NFR-03)	—	≥ 95%	≥ 90%	≥ 90%	< 1 s

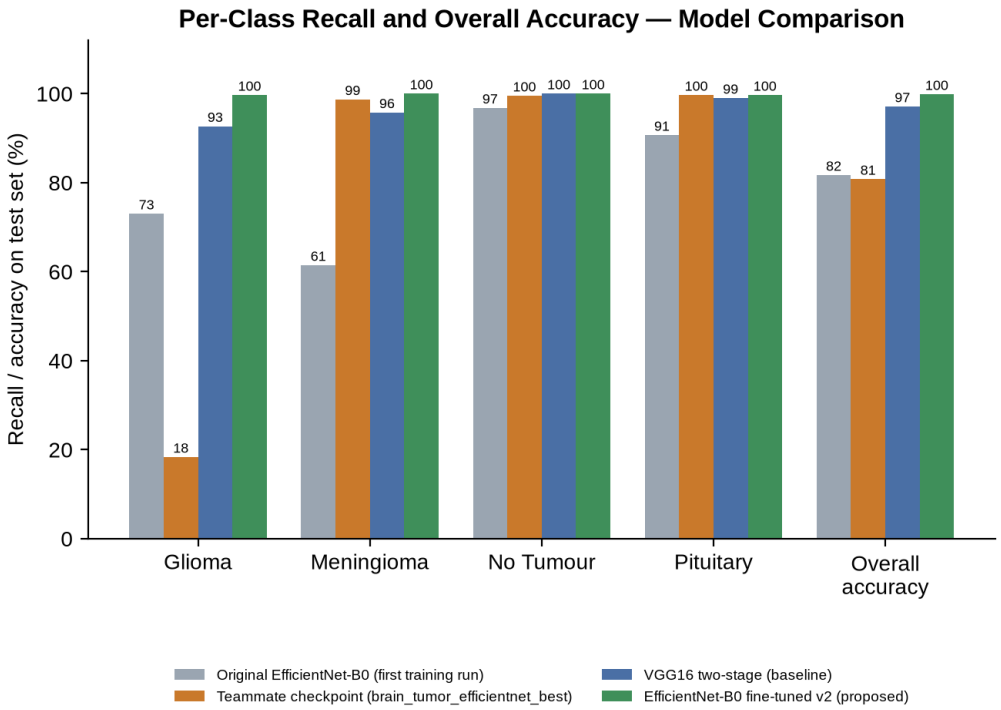


Fig 7.5: Per-Class Recall and Overall Accuracy — Model Comparison

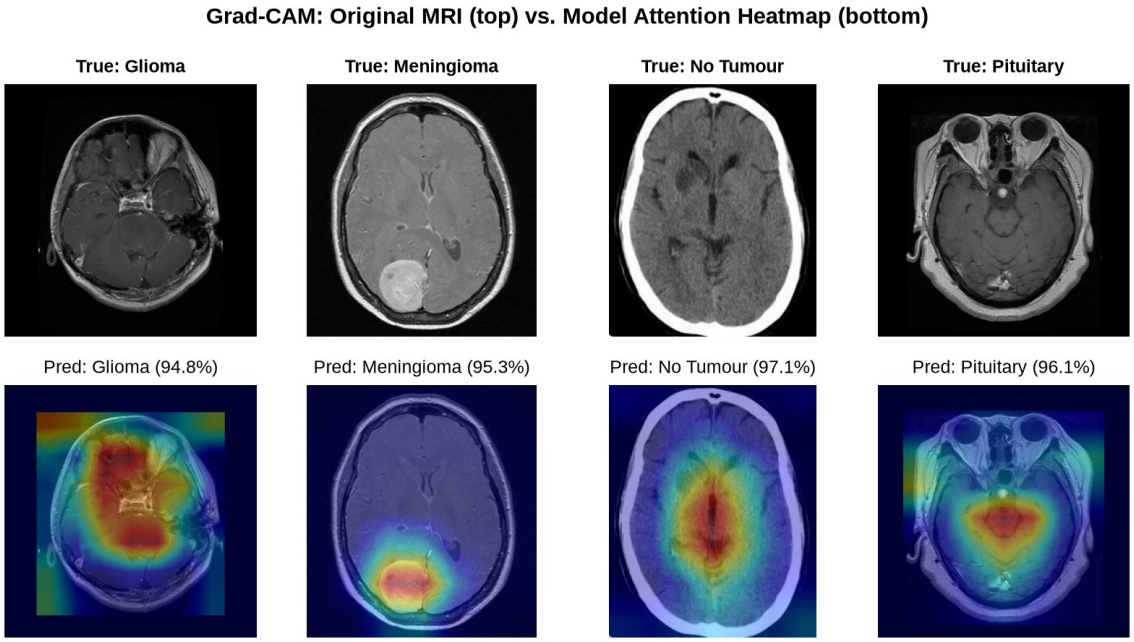


Fig 7.6: Grad-CAM Examples: Original MRI (top) and Model Attention Heatmap (bottom)

Fig 7.6 shows Grad-CAM maps of the final model for one test image of each class. For the tumour classes the maps concentrate on the lesion or its immediate surroundings; for the healthy scan the attention is spread over the central brain structures. These maps are an aid to inspection rather than proof of correct reasoning, but they let a user quickly spot a prediction whose highlighted area does not match the visible abnormality.

7.12 Screenshots of Testing Outcomes

The following figures show the raw output of the tests and checks described above.

```
pytest backend/tests -v

============================= test session starts ==============================
platform linux -- Python 3.14.8, pytest-9.1.1, pluggy-1.6.0 -- /home/crat/Projects/vedika/venv/bin/python
cachedir: .pytest_cache
rootdir: /home/crat/Projects/vedika
collecting ... collected 18 items

backend/tests/test_backend.py::test_crop_bbox_finds_bright_region PASSED [  5%]
backend/tests/test_backend.py::test_crop_bbox_all_black_returns_full_image PASSED [ 11%]
backend/tests/test_backend.py::test_crop_bbox_padding_is_clamped_to_image PASSED [ 16%]
backend/tests/test_backend.py::test_overlay_keeps_image_shape_and_dtype PASSED [ 22%]
backend/tests/test_backend.py::test_allowed_file_extensions PASSED       [ 27%]
backend/tests/test_backend.py::test_model_has_four_class_head PASSED     [ 33%]
backend/tests/test_backend.py::test_extracted_checkpoint_folder_loads PASSED [ 38%]
backend/tests/test_backend.py::test_health_reports_model_loaded PASSED   [ 44%]
backend/tests/test_backend.py::test_model_info_lists_classes_and_metrics PASSED [ 50%]
backend/tests/test_backend.py::test_predict_returns_correct_class_with_heatmap[glioma] PASSED [ 55%]
backend/tests/test_backend.py::test_predict_returns_correct_class_with_heatmap[meningioma] PASSED [ 61%]
backend/tests/test_backend.py::test_predict_returns_correct_class_with_heatmap[notumor] PASSED [ 66%]
backend/tests/test_backend.py::test_predict_returns_correct_class_with_heatmap[pituitary] PASSED [ 72%]
backend/tests/test_backend.py::test_predict_rejects_missing_file PASSED  [ 77%]
backend/tests/test_backend.py::test_predict_rejects_wrong_extension PASSED [ 83%]
backend/tests/test_backend.py::test_predict_rejects_corrupt_image PASSED [ 88%]
backend/tests/test_backend.py::test_predict_handles_tiny_all_black_image PASSED [ 94%]
backend/tests/test_backend.py::test_predict_rejects_oversized_upload_with_json_error PASSED [100%]

============================== 18 passed in 2.43s ==============================
```

Fig 7.7: Automated Backend Test Run — 18 Tests Passing

```
python ml/evaluate_api.py

   ...
              precision    recall  f1-score   support

      glioma       1.00      1.00      1.00       300
  meningioma       0.99      1.00      1.00       306
     notumor       1.00      1.00      1.00       405
   pituitary       1.00      1.00      1.00       300

    accuracy                           1.00      1311
   macro avg       1.00      1.00      1.00      1311
weighted avg       1.00      1.00      1.00      1311

=== Confusion matrix (rows=true, cols=pred) ===
              glioma  meningioma     notumor   pituitary
    glioma        299           1           0           0
meningioma          0         306           0           0
   notumor          0           0         405           0
 pituitary          0           1           0         299

Saved to /home/crat/Projects/vedika/model/efficientnet_v2/api_evaluation_report.json

Live API accuracy:   99.85%
Offline eval accuracy: 99.85% (from ml/train.py's own held-out evaluation)
```

Fig 7.8: Full Test Set through the Live API — 99.85% Accuracy

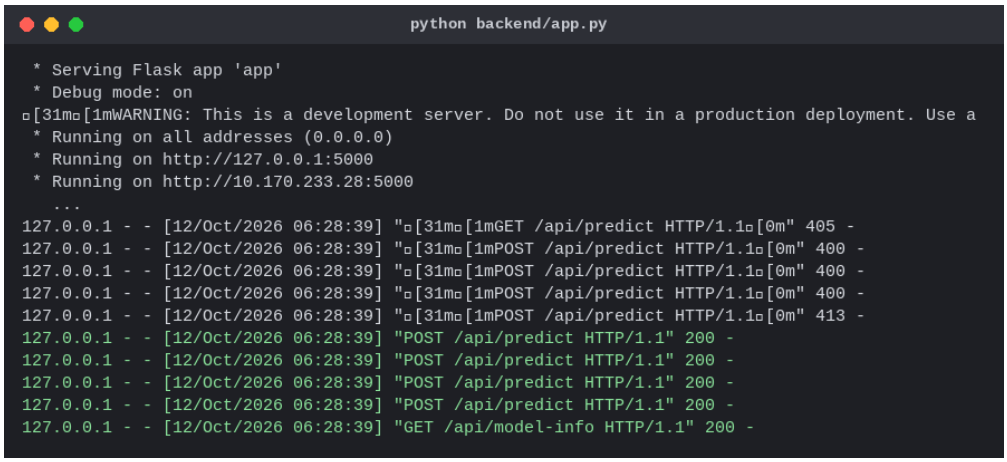


Fig 7.9: Backend Log while Serving Requests


```
{
  "backend": "efficientnet",
  "error": null,
  "model_loaded": true,
  "status": "ok"
}
```


Fig 7.10: Health Endpoint (/api/health) in the Browser

```
{
  "backend": "efficientnet",
  "classes": [
    "glioma",
    "meningioma",
    "notumor",
    "pituitary"
  ],
  "metrics": {
    "accuracy": 0.9984744469870328,
    "glioma": {
      "f1-score": 0.998330550918197,
      "precision": 1.0,
      "recall": 0.9966666666666667,
      "support": 300.0
    },
    "macro avg": {
      "f1-score": 0.9983509432115415,
      "precision": 0.9983766233766234,
      "recall": 0.9983333333333333,
      "support": 1311.0
    },
    "meningioma": {
      "f1-score": 0.996742671009772,
      "precision": 0.9935064935064936,
      "recall": 1.0,
      "support": 306.0
    },
    "notumor": {
      "f1-score": 1.0,
      "precision": 1.0,
      "recall": 1.0,
      "support": 405.0
    },
    "pituitary": {
      "f1-score": 0.998330550918197,
      "precision": 1.0,
      "recall": 0.9966666666666667,
      "support": 300.0
    },
    "weighted avg": {
      "f1-score": 0.998475658184522,
      "precision": 0.9984843531754286,
      "recall": 0.9984744469870328,
      "support": 1311.0
    }
  }
}
```


Fig 7.11: Model-information Endpoint (/api/model-info) in the Browser

Chapter 8
Conclusion and Future Work

8.1 Summary of the Project

This project set out to build an accurate and explainable system for classifying brain MRI slices into glioma, meningioma, pituitary tumour and no tumour, and to deliver it as a web application that behaves reliably. The system that resulted consists of a React interface, a Flask API and a PyTorch inference layer that serves a fine-tuned EfficientNet-B0 model, with a VGG16 model kept as an alternative back-end. Every prediction is returned with the confidence of the four classes and a Grad-CAM heatmap, and the whole path from browser to model has been tested automatically and under load.

The most instructive part of the work was the repair of the model. A checkpoint that reported about 81% accuracy detected only 18% of gliomas. By measuring per-class recall, ruling out the serving path, and then fine-tuning with a higher learning rate, plain weighted cross-entropy and a checkpoint rule that protects the worst class, the same network reached 99.85% accuracy on the held-out test set and at least 99.7% recall in every class.

8.2 Achievements

- Built and compared three classifiers, and delivered a fine-tuned EfficientNet-B0 with **99.85% test accuracy** (1,309 of 1,311 images), compared with 97.10% for the VGG16 baseline and 80.8% for the starting checkpoint.
- Raised **glioma recall from 18% to 99.7%** and ensured that no class has a recall below 99.7%.
- Integrated **Grad-CAM** so that every prediction carries a visual explanation.
- Reduced the model to **4.0 million parameters and a 16 MB checkpoint**, and the response time to about **0.15 s** per scan on a CPU, which is roughly seventeen times faster than the VGG16 back-end.
- Verified that the **live API reproduces the offline evaluation exactly** (99.85% on all 1,311 images, no failed requests).
- Wrote **18 automated tests** and a latency benchmark; found and fixed a **concurrency crash** (inference lock) and an **oversized-upload error format**, and made the server's defaults safer (debug off, localhost binding).

- Made the loader accept a checkpoint that had been unzipped into a folder, so the team-mate's file could be used unchanged.
- Produced reproducible scripts for training, evaluation, benchmarking and figure generation.

8.3 Comparison of Objectives vs Outcomes

Table 8.1: Comparison of Objectives vs Outcomes

Objective	Target	Outcome	Met?
Prepare a consistent pre-processing pipeline	Crop, resize, normalise; same in training and serving	Shared definition; API result equals offline result	Yes
Overall accuracy	≥ 95%	99.85% (95% CI 99.45–99.96%)	Yes
Recall of every class	≥ 90%	Lowest class recall 99.7%	Yes
Compare two transfer-learning models	VGG16 and EfficientNet-B0	Four models compared on one test set (Table 7.7)	Yes
Explainability	Heatmap for every prediction	Grad-CAM overlay returned by the API and shown with a toggle	Yes
Responsive service	P95 latency < 1 s	P95 0.20 s (one user), 0.41 s (four users)	Yes
Robust input handling	No crash on invalid input	400 / 413 JSON errors for all invalid cases tested	Yes
Verification	Tests, full-API run, benchmark	18 tests, 1,311-image API run, 100-request benchmarks	Yes
Clinical validity	Not an objective	Not assessed; disclaimer displayed	Out of scope

8.4 Current Limitations

- **Two-dimensional slices only.** The system classifies single slices; it does not combine the slices of a study or use several MRI sequences, which a radiologist would do.
- **Single public dataset.** All results come from one dataset. Performance on scans from other scanners, protocols or hospitals has not been measured and is likely to be lower, particularly if slices of the same patient appear on both sides of the dataset's split.
- **No rejection of unsuitable images.** Any decodable image receives one of the four labels, including non-MRI pictures.
- **Four classes only.** Other tumour types, and combinations such as a tumour together with other pathology, are not represented.
- **Classification, not localisation.** Grad-CAM maps are coarse (7×7 resolution) and approximate; the system does not segment the tumour or measure its size.

- **Confidence is not calibrated.** The softmax probability was not checked against observed accuracy, so a displayed 97% should not be read as a 97% chance of being correct.
- **Prototype deployment.** The server uses Flask's development server, has no authentication or rate limiting and keeps results only in the browser session.
- **Single training run.** The reported figures come from one run on one machine; variance across seeds was not measured.
- **Not clinically validated.** The application is for education and research and must not be used for diagnosis.

8.5 Future Work — External Validation and Data Diversity

The first priority is to test the model on data it could not have seen in any form: scans from another hospital or from a different public collection, with patient-level splits so that no patient contributes to both training and test data. Where performance falls, the model can be retrained on a mixture of sources with stronger augmentation. Repeating the training with several random seeds and reporting the spread would also turn the single accuracy figure into a reliable estimate.

8.6 Future Work — Multi-sequence, 3-D Analysis and Segmentation

Radiologists read a tumour through several sequences and many slices. A natural extension is to accept a full study, classify each slice, and combine the slice predictions into one decision, or to move to three-dimensional networks. Adding a segmentation model such as a U-Net trained on BraTS [15] would allow the system to outline the tumour and to report its volume, replacing the coarse Grad-CAM region with an explicit mask.

8.7 Future Work — Reliability, Calibration and Out-of-distribution Detection

To make the confidence meaningful, the probabilities can be calibrated, for example with temperature scaling on the validation set, and reliability diagrams reported. An out-of-distribution check — based on the entropy of the prediction, on feature-space distance from the training data, or on a small classifier trained to recognise brain MRI slices — would let the system answer "this does not look like a brain MRI" instead of guessing. Ensembles of the EfficientNet and VGG16 back-ends, or test-time augmentation, could further reduce the rare errors between glioma, meningioma and pituitary tumours.

8.8 Future Work — Clinical Integration and Deployment

For use beyond demonstration the application would be packaged in a container and served by a production WSGI server behind a reverse proxy with HTTPS, authentication, rate limiting and audit logging. Medical interoperability would require reading DICOM files, de-identifying them, and exchanging results with PACS and electronic health records through DICOM and HL7/FHIR. Regulatory and privacy requirements such as HIPAA and GDPR, and a prospective clinical study with radiologists, would be prerequisites for any real-world use. A mobile or desktop client that reuses the same API is also possible without changing the back end.

References

[1] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv preprint arXiv:1409.1556, 2014; presented at ICLR, 2015.

[2] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in Proc. 36th Int. Conf. on Machine Learning (ICML), 2019, pp. 6105–6114.

[3] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” in Proc. IEEE Int. Conf. on Computer Vision (ICCV), 2017, pp. 618–626.

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” in Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2009, pp. 248–255.

[5] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in Proc. IEEE CVPR, 2016, pp. 770–778.

[6] A. G. Howard et al., “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” arXiv preprint arXiv:1704.04861, 2017.

[7] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, “Focal Loss for Dense Object Detection,” in Proc. IEEE ICCV, 2017, pp. 2980–2988.

[8] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv preprint arXiv:1412.6980, 2014; presented at ICLR, 2015.

[9] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” arXiv preprint arXiv:1711.05101, 2017; presented at ICLR, 2019.

[10] L. N. Smith and N. Topin, “Super-Convergence: Very Fast Training of Neural Networks Using Very Large Learning Rates,” arXiv preprint arXiv:1708.07120, 2017.

[11] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

[12] M. Abadi et al., “TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems,” arXiv preprint arXiv:1603.04467, 2016.

[13] M. Nickparvar, “Brain Tumor MRI Dataset,” Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/masoudnickparvar/brain-tumor-mri-dataset

[14] J. Cheng et al., “Enhanced Performance of Brain Tumor Classification via Tumor Region Augmentation and Partition,” PLOS ONE, vol. 10, no. 10, e0140381, 2015.

[15] U. Baid et al., “The RSNA-ASNR-MICCAI BraTS 2021 Benchmark on Brain Tumor Segmentation and Radiogenomic Classification,” arXiv preprint arXiv:2107.02314, 2021.

[16] M. Antonelli et al., “The Medical Segmentation Decathlon,” Nature Communications, vol. 13, art. 4128, 2022.

[17] D. N. Louis et al., “The 2021 WHO Classification of Tumors of the Central Nervous System: A Summary,” Neuro-Oncology, vol. 23, no. 8, pp. 1231–1251, 2021.

[18] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[19] C. R. Harris et al., “Array Programming with NumPy,” Nature, vol. 585, pp. 357–362, 2020.

[20] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, 2000.

[21] Pallets Projects, “Flask Documentation,” 2024. [Online]. Available: https://flask.palletsprojects.com

[22] Meta Platforms, Inc., “React Documentation,” 2024. [Online]. Available: https://react.dev

[23] J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.